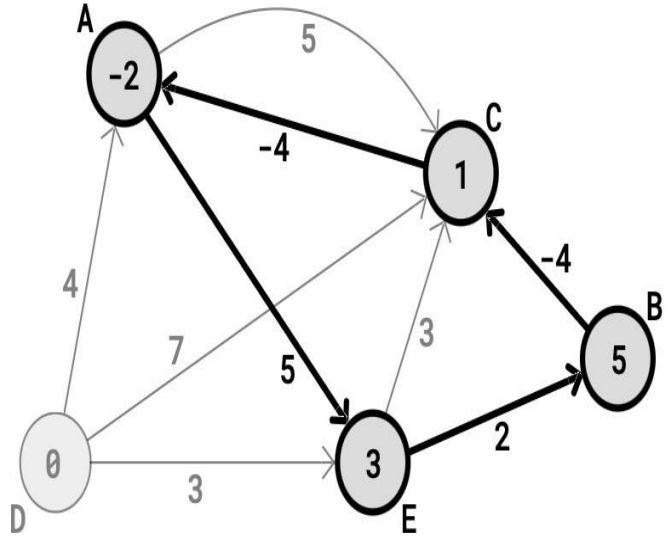
2.4 Advanced Shortest Path Algorithms

1. **Bellman-Ford Algorithm**
2. **Johnson's algorithm**

Shortest Path Algorithm

- Algorithms that find the shortest paths, like Dijkstra's algorithm or the Bellman-Ford algorithm, find the shortest paths from one start vertex to all other vertices.
- Dijkstra's algorithm can only find the shortest paths in graphs where all the edges are positive. We can think of the edges between vertices as distances between locations.
- We can represent negative edge for money lost situation.
- Finding the shortest paths becomes impossible if a graph has negative cycles.
- Having a negative cycle means that there is a path where you can go in circles, and the edges that make up this circle have a total path weight that is negative.
- In the graph below, the path A->E->B->C->A is a negative cycle because the total path weight is 5+2-4-4=-1.
- The reason why it is impossible to find the shortest paths in a graph with negative cycles is that it will always be possible to continue running an algorithm to find even shorter paths.



Shortest Path Algorithm

Bellman-Ford Algorithm

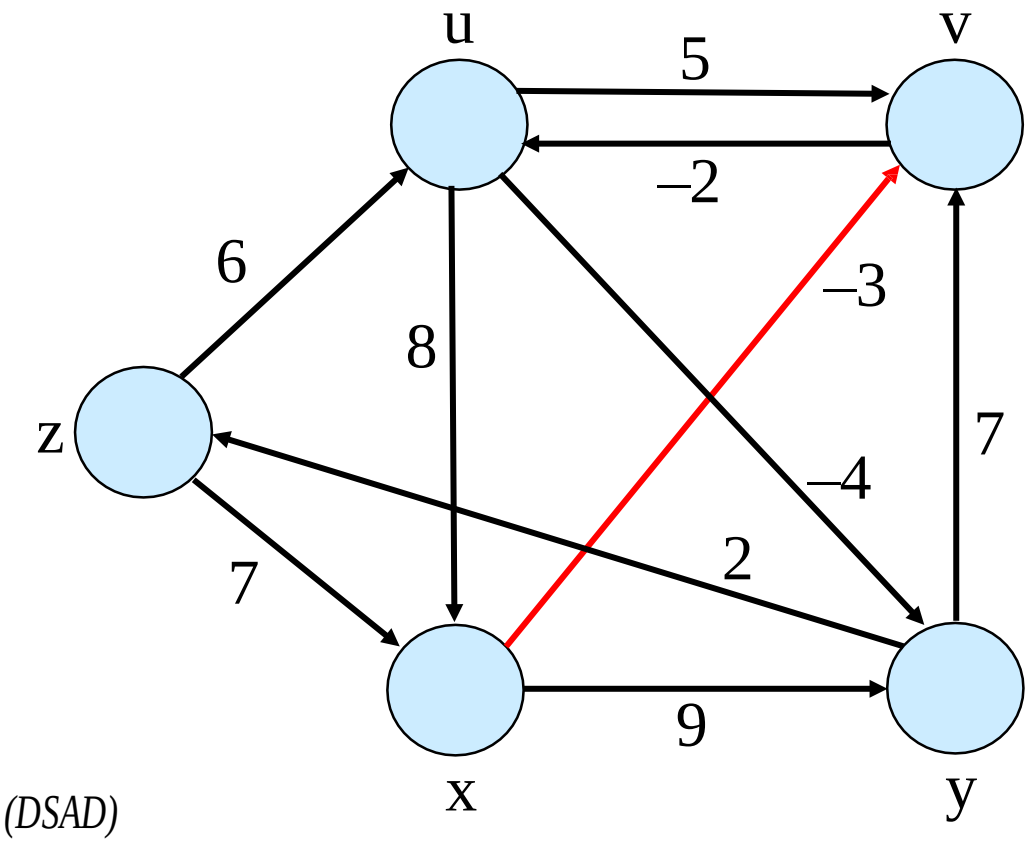
- The Bellman-Ford algorithm solves the single-source shortest-paths problem in the general case in which edge weights may be negative.
- Given a weighted, directed graph G = (V, E) with source vertex *s* and weight function w:E→ R, the Bellman-Ford algorithm returns a boolean value indicating whether there is a negative-weight cycle that is reachable from the source.
- If there is such a cycle, the algorithm indicates that no solution exists. If there is no such cycle, the algorithm produces the shortest paths and their weights.

Single-Source Shortest Paths

- **Given:** A single source vertex in a weighted, directed graph.
- Want to compute a shortest path for each possible destination.
  - Similar to BFS.
- We will assume either
  - no negative-weight edges, or
  - no reachable negative-weight cycle that is reachable from source.
- Algorithm will compute a shortest-path tree.
  - Similar to BFS tree.
  - If there is a negative weight cycle, the algorithm indicates that no solution exists.
  - If there is no such cycle, the algorithm produces the shortest paths and their weights.

Negative Edge

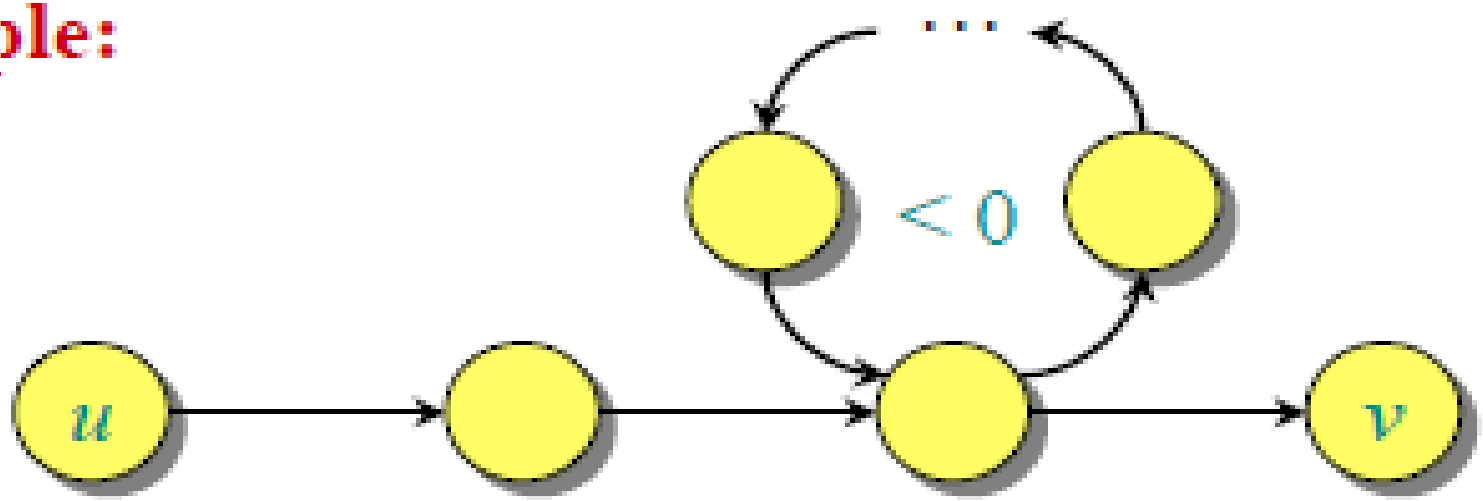
- A negative edge is simply an edge having a negative weight.
  - For example, the edge X-V in the graph is a negative edge. For a negative weight you could simply perform the calculation as you would have done for positive weight edges.



Negative-weight cycles

Recall: If a graph $G = (V, E)$ contains a negative-weight cycle, then some shortest paths may not exist.

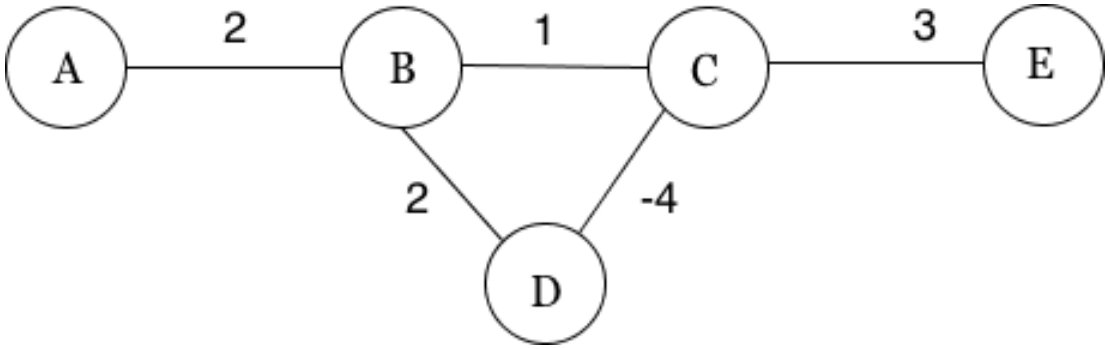
Example:



Bellman-Ford algorithm: Finds all shortest-path lengths from a ***source*** $s \in V$ to all $v \in V$ or determines that a negative-weight cycle exists.

Negative Weight Cycle..

- What is the shortest path between A and E?
  - Initially, we can see that ABCE costing 6 (2+1+3). But actually, taking a deeper look, we would observe a negative cycle, which is BCD.The weight of BCD is 1+(-4)+2 = (-1).
  - While traversing from A to E, it is possible to looping around inside BCD to reduce cost by 1 each time. Like, the path A(BCD)BCE costs 5 (2+(-1)+1+3).
  - Now repeating the cycle infinite times would keep reducing the cost by 1 each time.We could achieve a negative infinite shortest path between A and E.
  - The problem is evident for any negative cycle in a graph. Hence, whenever a negative cycle is present, the minimum weight is not defined or is negative infinity.



Bellman-Ford Algorithm

Bellman-Ford returns a compact representation of the set of shortest paths from s to all other vertices in the graph reachable from s. This is contained in the predecessor subgraph.

If Bellman-Ford has not converged after V(G) - 1 iterations, then there cannot be a shortest path tree, so there must be a negative weight cycle.

Bellman-Ford algorithm

```
d[s] ← 0
for each v ∈ V − {s}                    } initialization
    do d[v] ← ∞

for i ← 1 to |V| − 1 do
  for each edge (u, v) ∈ E do
    if d[v] > d[u] + w(u, v) then       } relaxation
          d[v] ← d[u] + w(u, v)           step
          π[v] ← u
for each edge (u, v) ∈ E
    do if d[v] > d[u] + w(u, v)
          then report that a negative-weight cycle exists
```

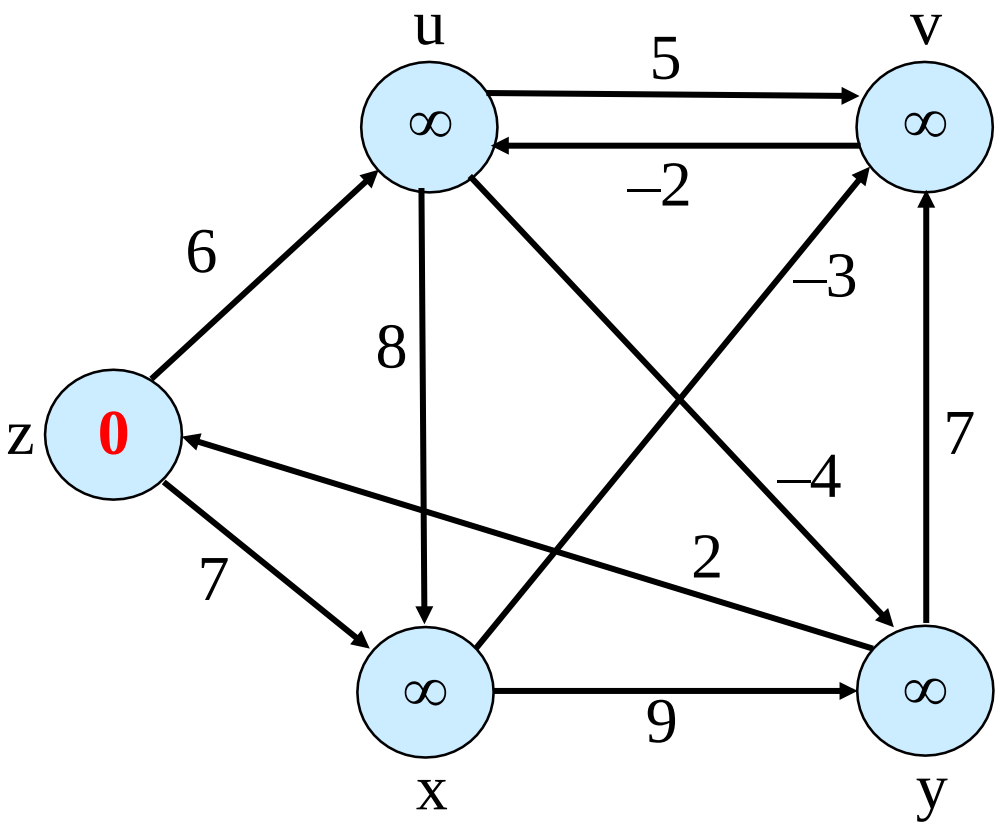
Can have negative-weight edges.

Will detect" reachable negative-weight cycles

Bellman-Ford Algorithm

Example

Order of Edges
(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)

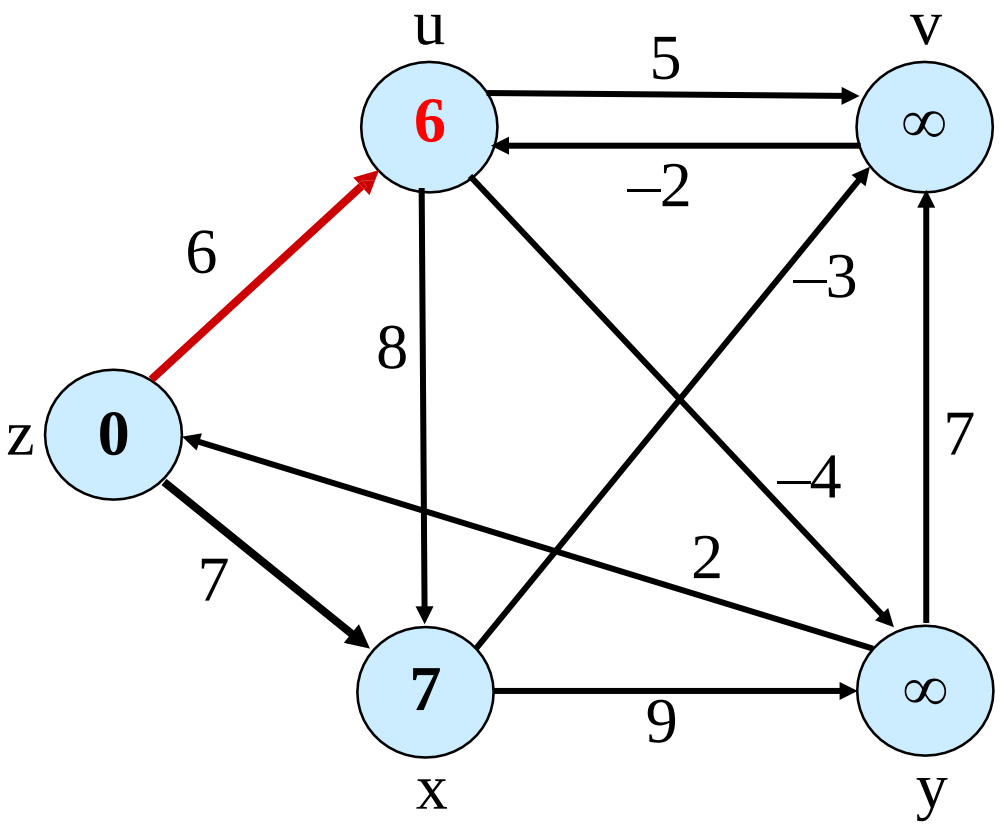


Vertex	D[v]	∏[v]
Z	0	Nil
U	∞	Nil
V	∞	Nil
X	∞	Nil
Y	∞	Nil

Bellman-Ford Algorithm

Example

Order of Edges
(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)



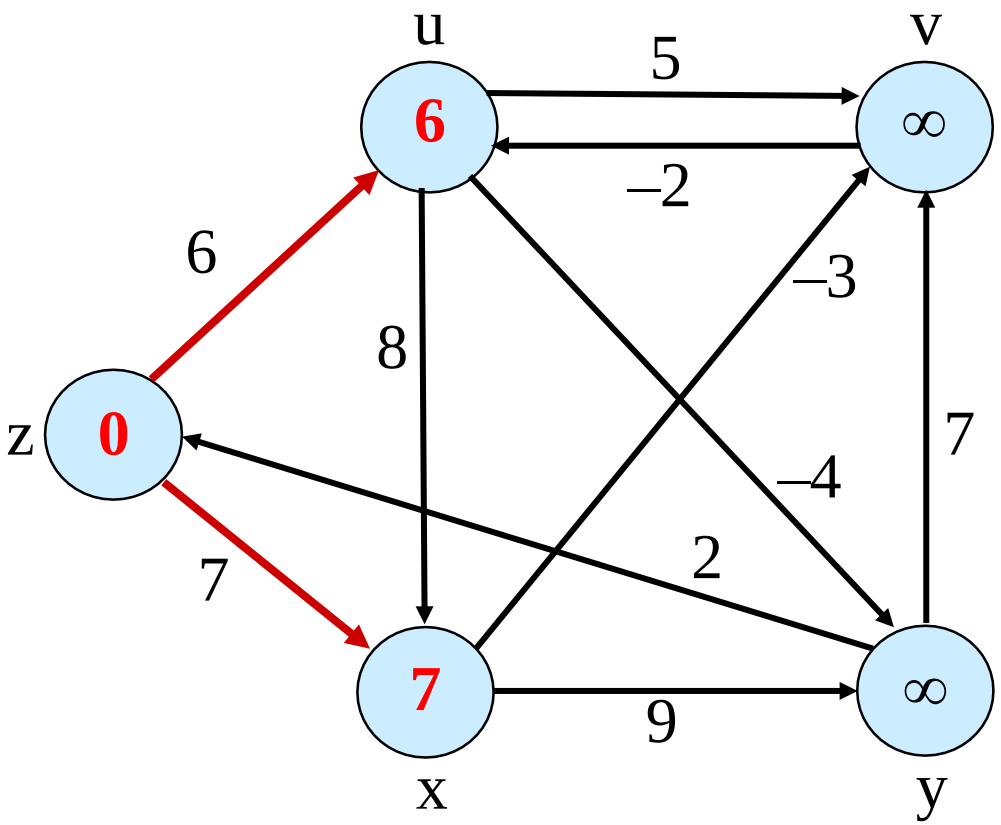
Vertex	D[v]	∏[v]
Z	0	Nil
U	6	z
V	∞	Nil
X	∞	Nil
Y	∞	Nil

Pass 1

Bellman-Ford Algorithm

Example

Order of Edges
(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)



Vertex	D[v]	∏[v]
Z	0	Nil
U	6	z
V	∞	Nil
X	7	z
Y	∞	Nil

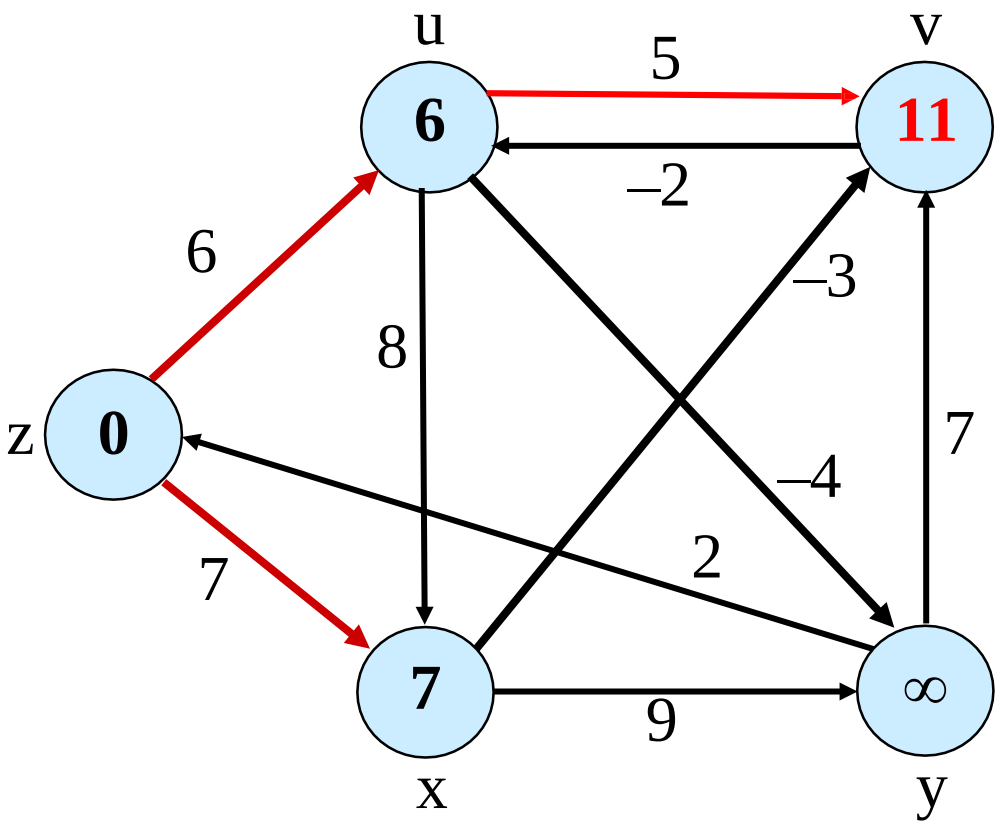
Pass 1

Bellman-Ford Algorithm

Example

Order of Edges

(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)



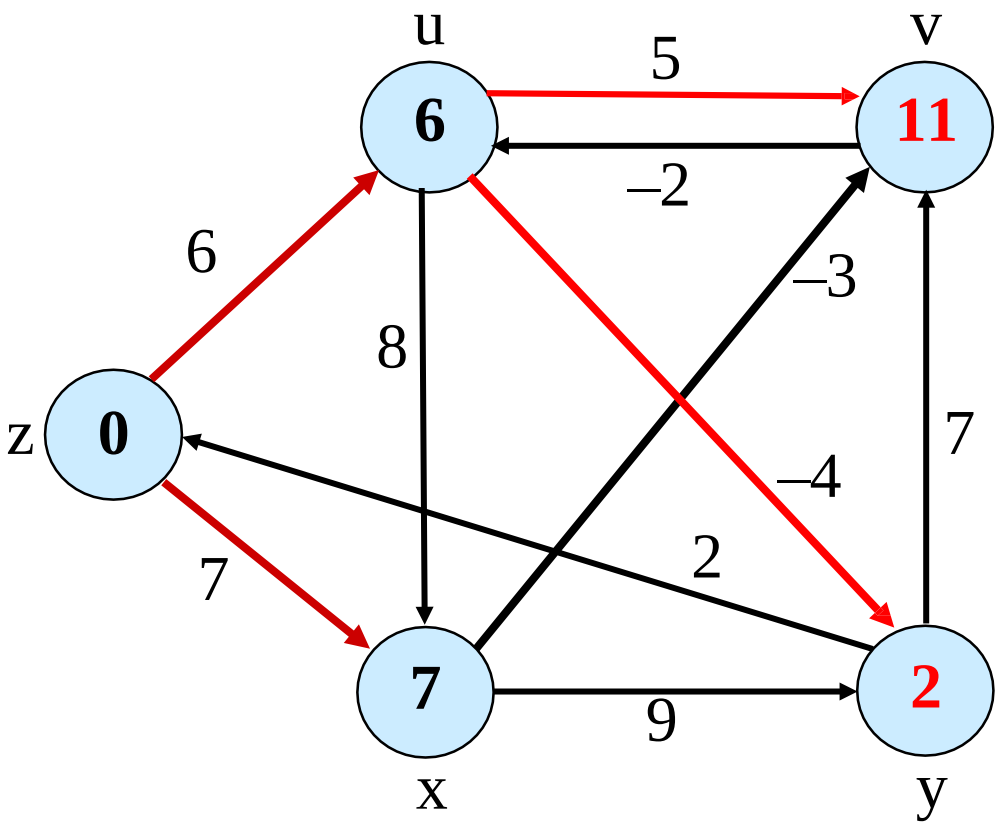
Vertex	D[v]	Π[v]
Z	0	Nil
U	6	z
V	11	U
X	7	z
Y	∞	Nil

Bellman-Ford Algorithm

Example

Order of Edges

(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)



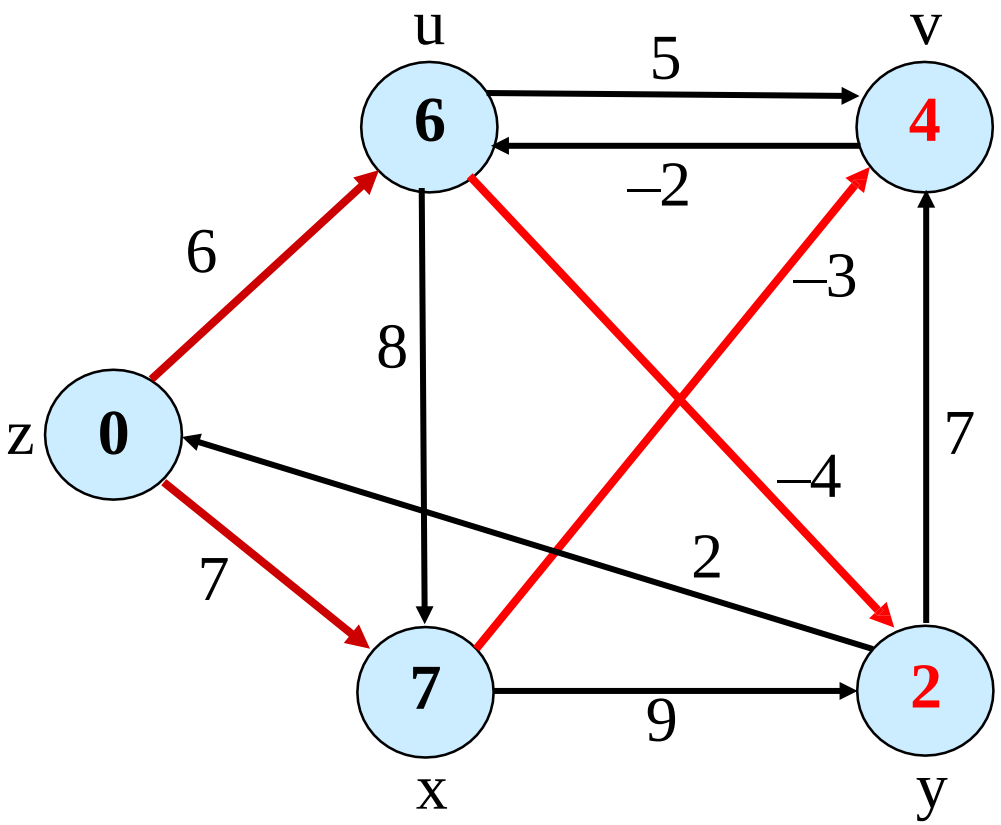
Vertex	D[v]	∏[v]
Z	0	Nil
U	6	z
V	11	U
X	7	z
Y	2	U

Bellman-Ford Algorithm

Example

Order of Edges

(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)

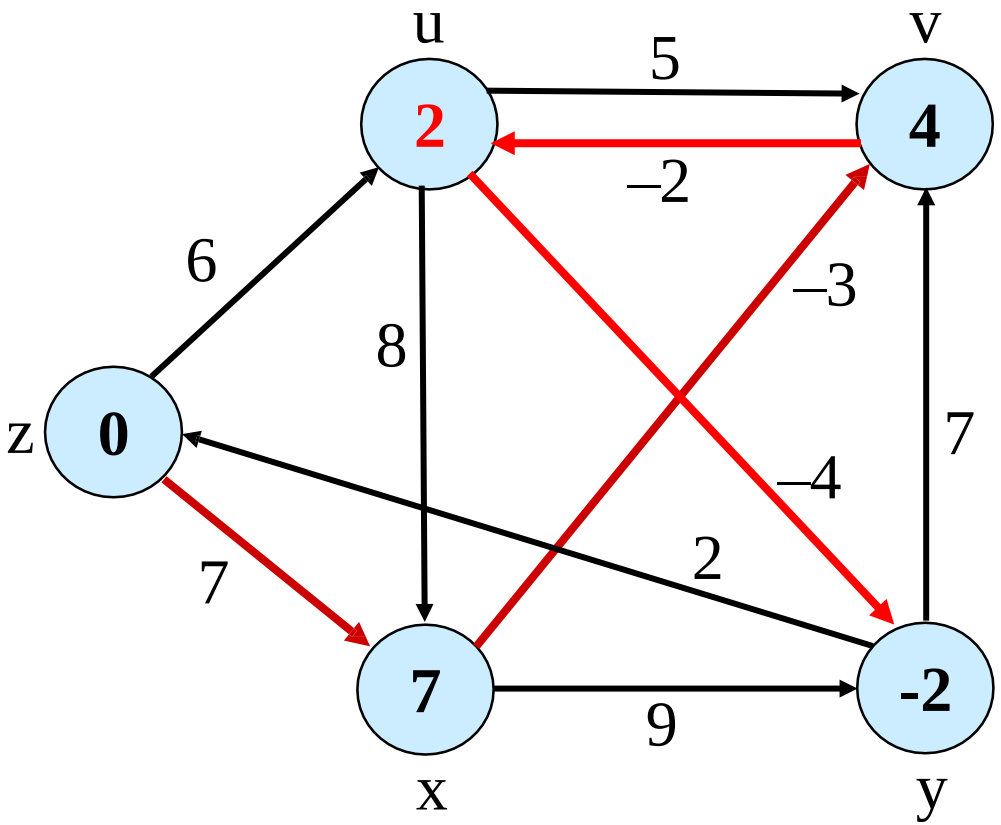


Vertex	D[v]	∏[v]
Z	0	Nil
U	6	z
V	4	x
X	7	z
Y	2	U

Bellman-Ford Algorithm

Example

Order of Edges
(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)



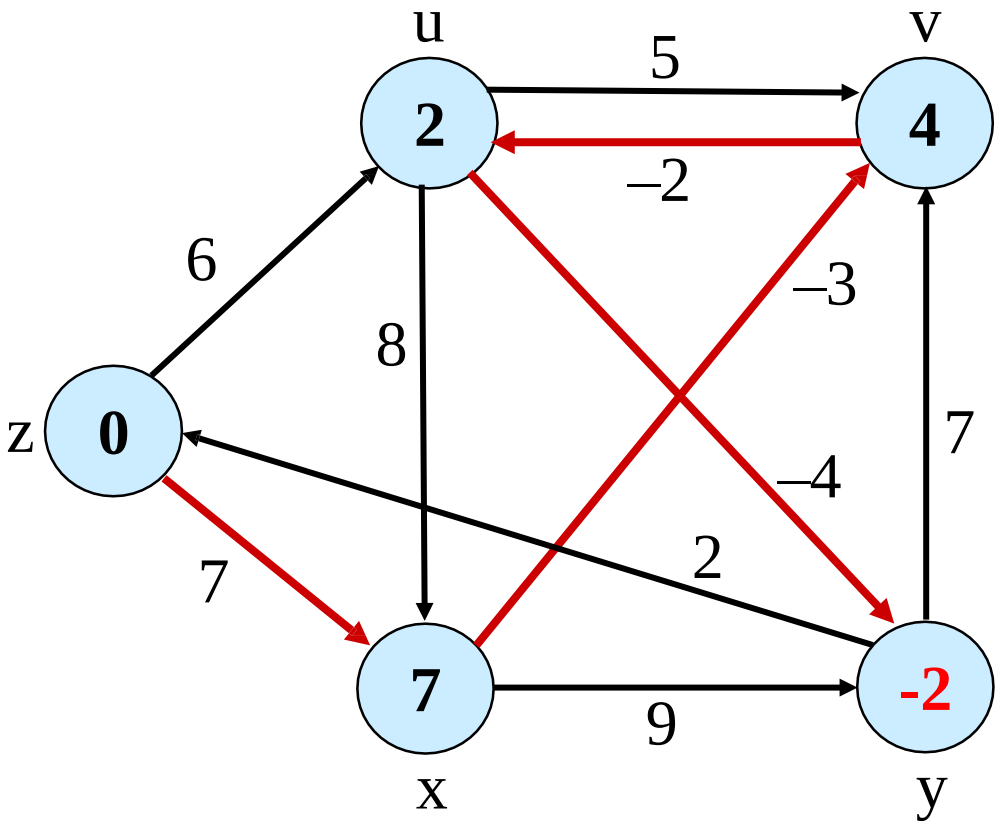
Vertex	D[v]	∏[v]
Z	0	Nil
U	2	v
V	4	x
X	7	z
Y	-2	U

Bellman-Ford Algorithm

Example

Order of Edges

(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)



Vertex	D[v]	∏[v]
Z	0	Nil
U	2	v
V	4	x
X	7	z
Y	-2	U

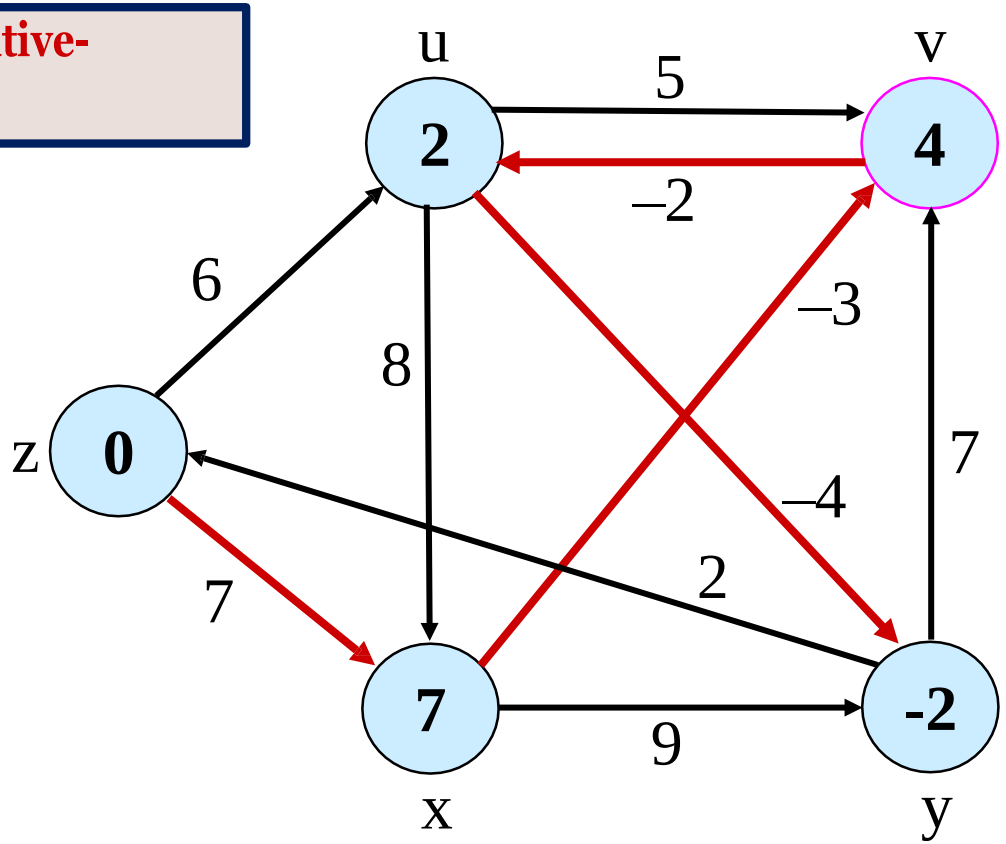
Pass 4

Bellman-Ford Algorithm

Example

Order of Edges
(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)

"detect" reachable negative-weight cycles.



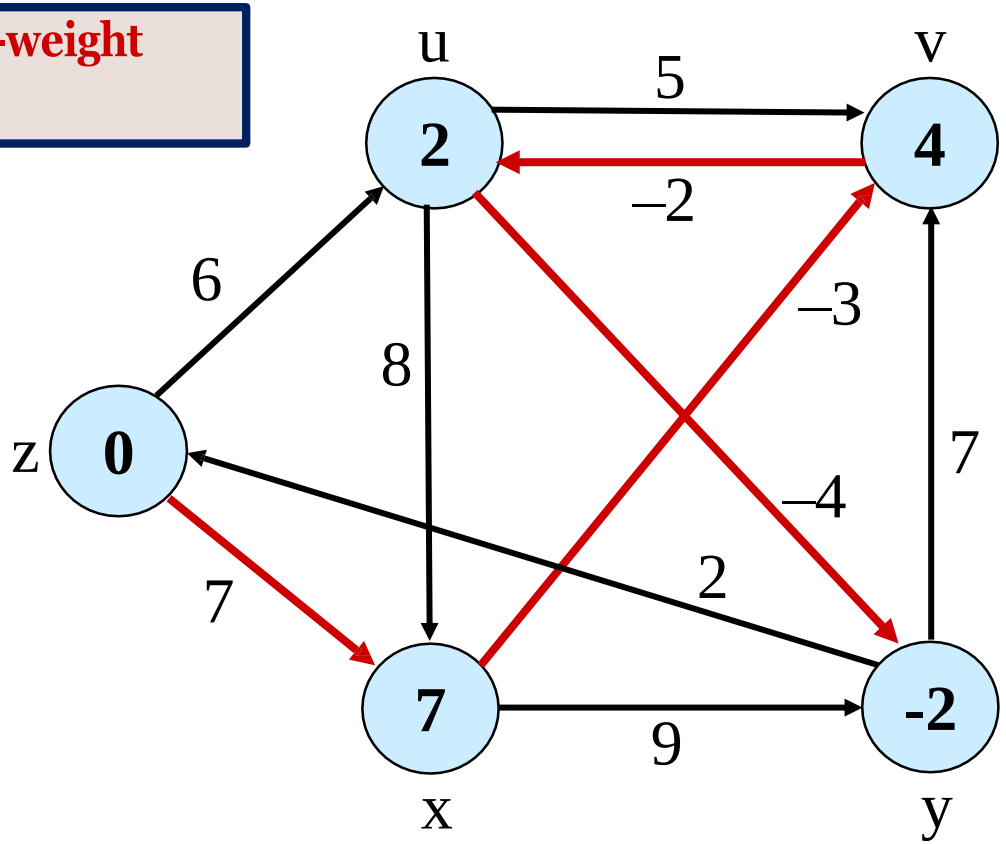
Vertex	D[v]	∏[v]
Z	0	Nil
U	2	v
V	4	x
X	7	z
Y	-2	U

Bellman-Ford Algorithm

Example

Order of Edges
(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)

No reachable negative-weight cycles.



Vertex	D[v]	∏[v]
Z	0	Nil
U	2	v
V	4	x
X	7	z
Y	-2	U

Bellman Ford Algorithm Analysis

```
d[s] ← 0
for each v ∈ V − {s}
    do d[v] ← ∞

for i ← 1 to |V| − 1 do
   for each edge (u, v) ∈ E do
      if d[v] > d[u] + w(u, v)  then
            d[v] ← d[u] + w(u, v)
            π[v] ← u
for each edge (u, v) ∈ E
    do if d[v] > d[u] + w(u, v)
          then  report that a negative-weight cycle exists
```

O(V)

O(1)

O(E)

O(VE)

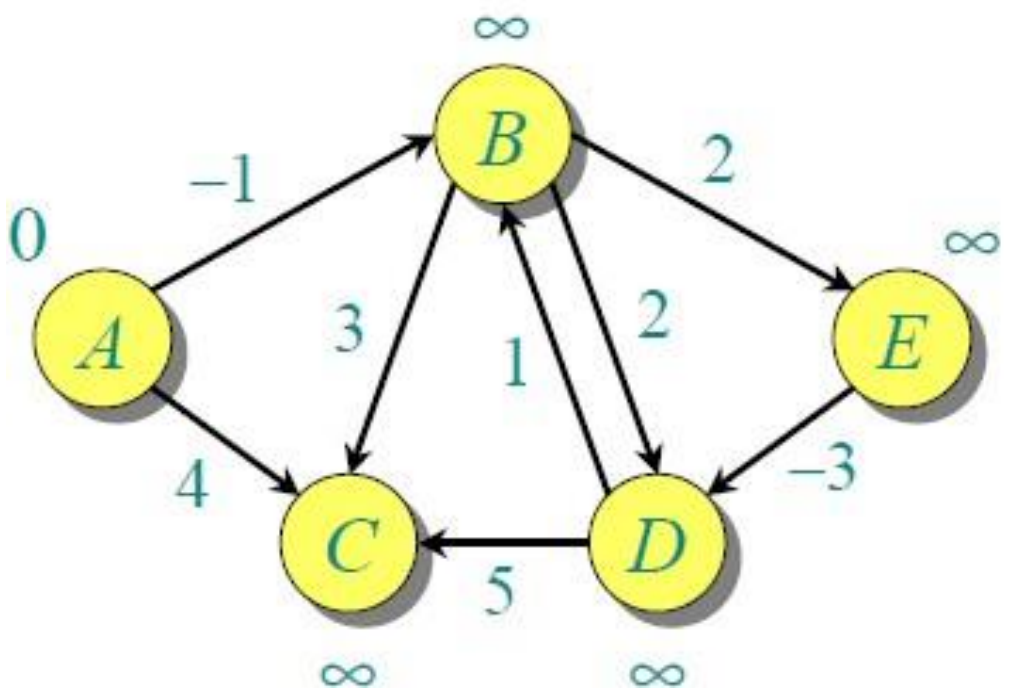
O(E)

At the end, $d[v] = \delta(s, v)$. Time = $O(|V|\,|E|)$.

Bellman-Ford Algorithm

Example of Bellman-Ford

Order of edges: *(B,E), (D,B), (B,D), (A,B), (A,C), (D,C), (B,C), (E,D)*



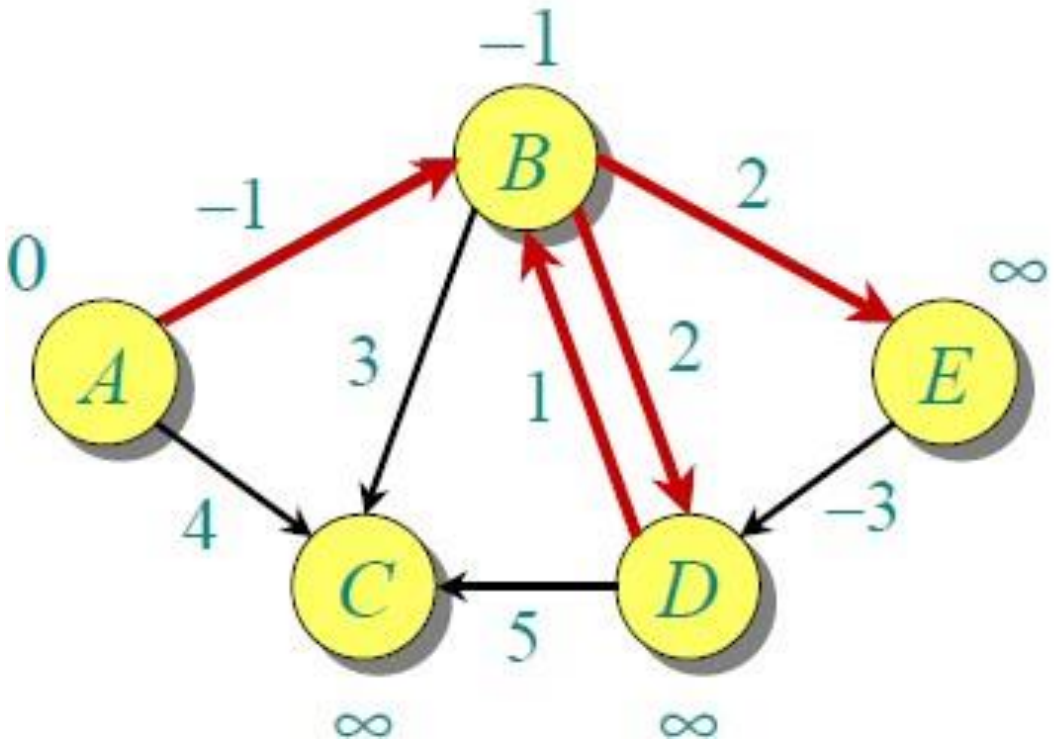
A	*B*	*C*	*D*	*E*
0	∞	∞	∞	∞

Repeat *V-1* times: relax all *E* edges.

Initialize

Example of Bellman-Ford

Order of edges: *(B,E), (D,B), (B,D), (A,B), (A,C), (D,C), (B,C), (E,D)*

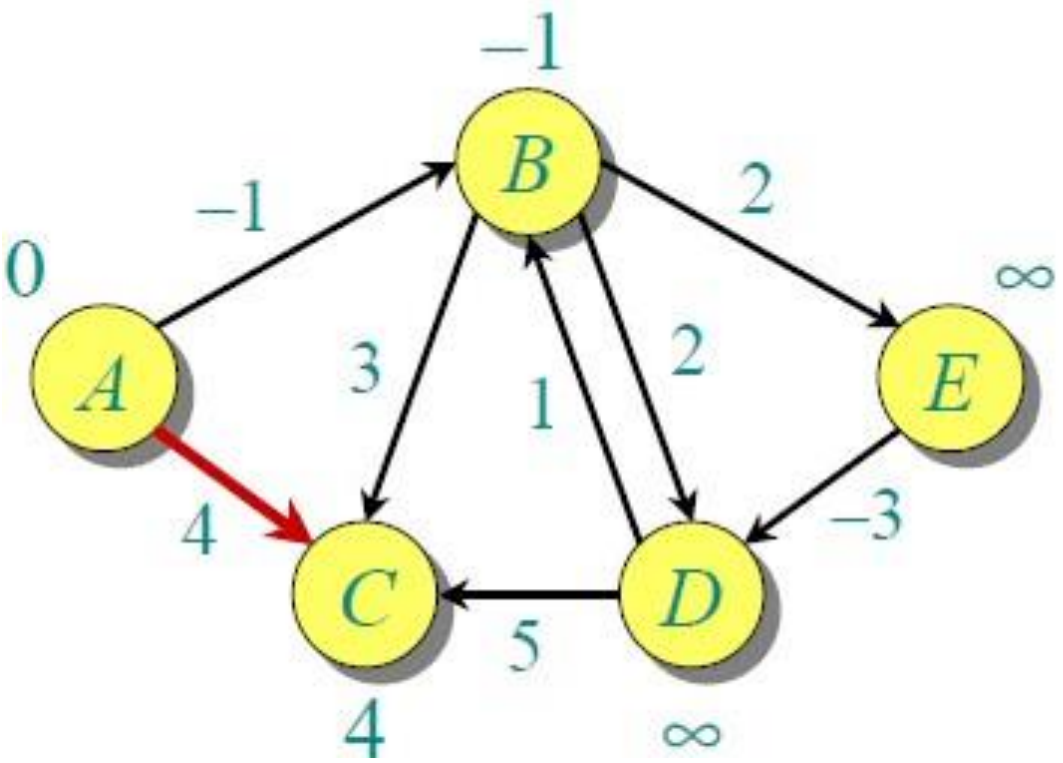


A	*B*	*C*	*D*	*E*
0	∞	∞	∞	∞
0	−1	∞	∞	∞

Pass 1

Example of Bellman-Ford

Order of edges: *(B,E), (D,B), (B,D), (A,B), (A,C), (D,C), (B,C), (E,D)*

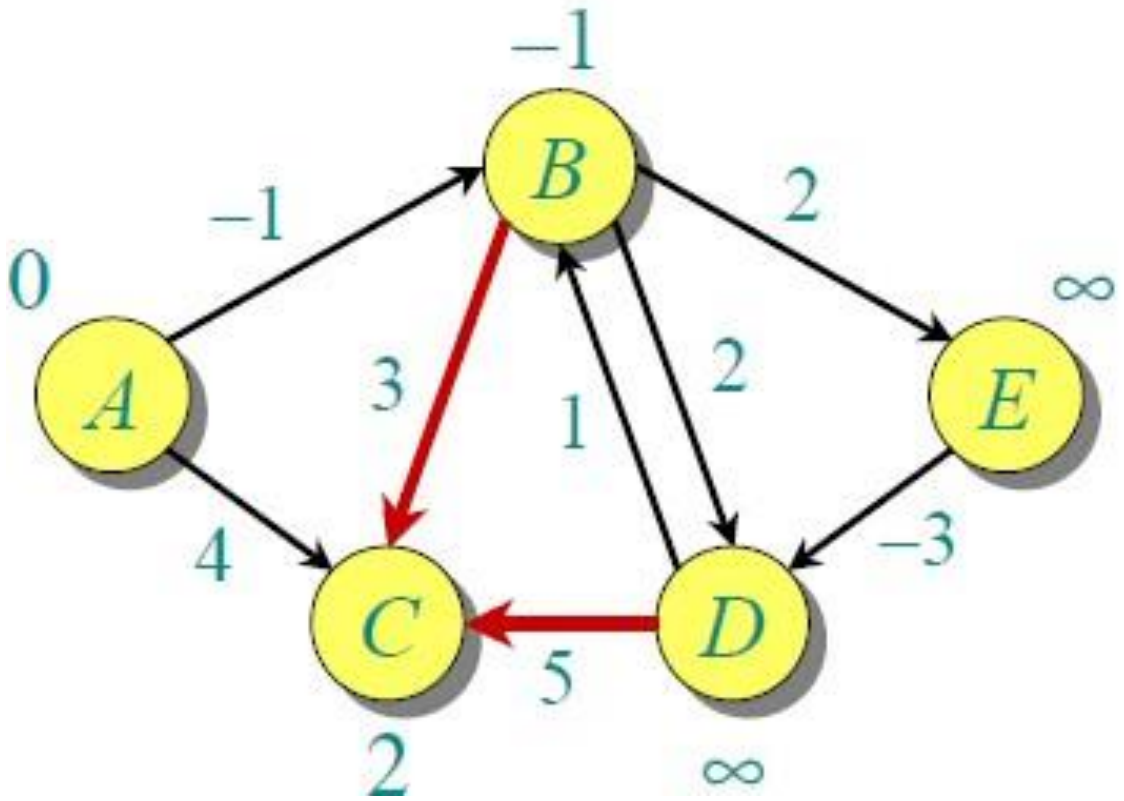


A	*B*	*C*	*D*	*E*
0	∞	∞	∞	∞
0	–1	∞	∞	∞
0	–1	4	∞	∞

Pass 1

Example of Bellman-Ford

Order of edges: *(B,E), (D,B), (B,D), (A,B), (A,C), (D,C), (B,C), (E,D)*

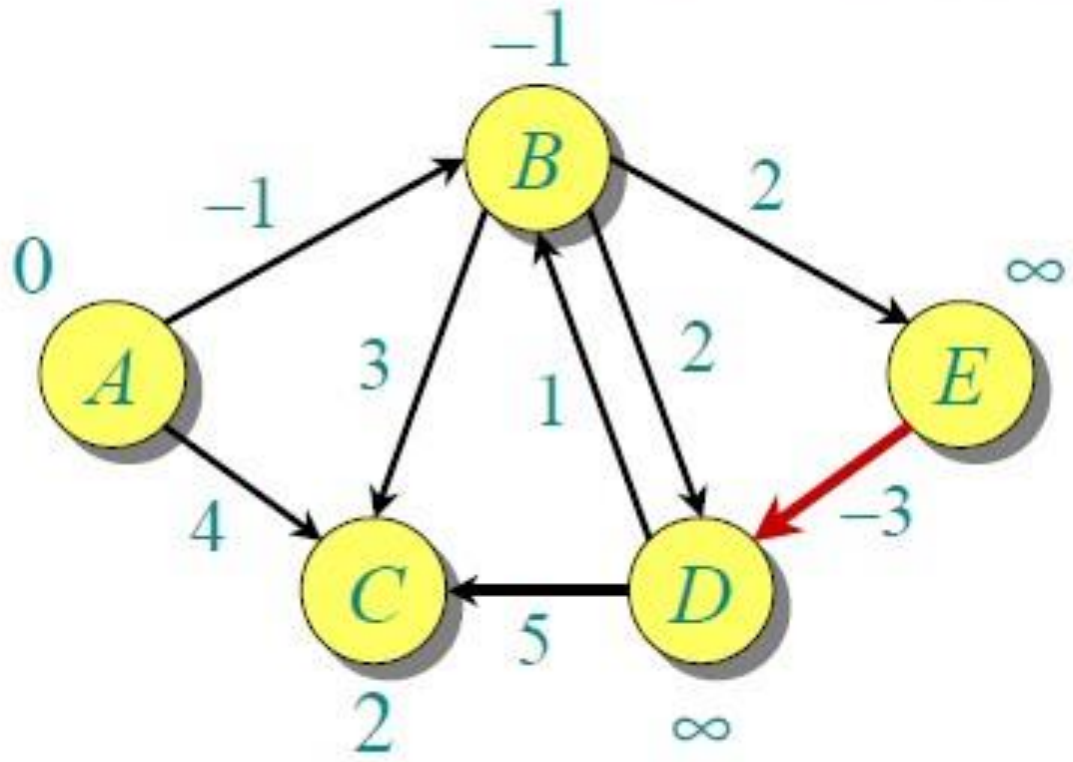


A	*B*	*C*	*D*	*E*
0	∞	∞	∞	∞
0	−1	∞	∞	∞
0	−1	4	∞	∞
0	−1	2	∞	∞

Pass 1

Example of Bellman-Ford

Order of edges: *(B,E), (D,B), (B,D), (A,B), (A,C), (D,C), (B,C), (E,D)*

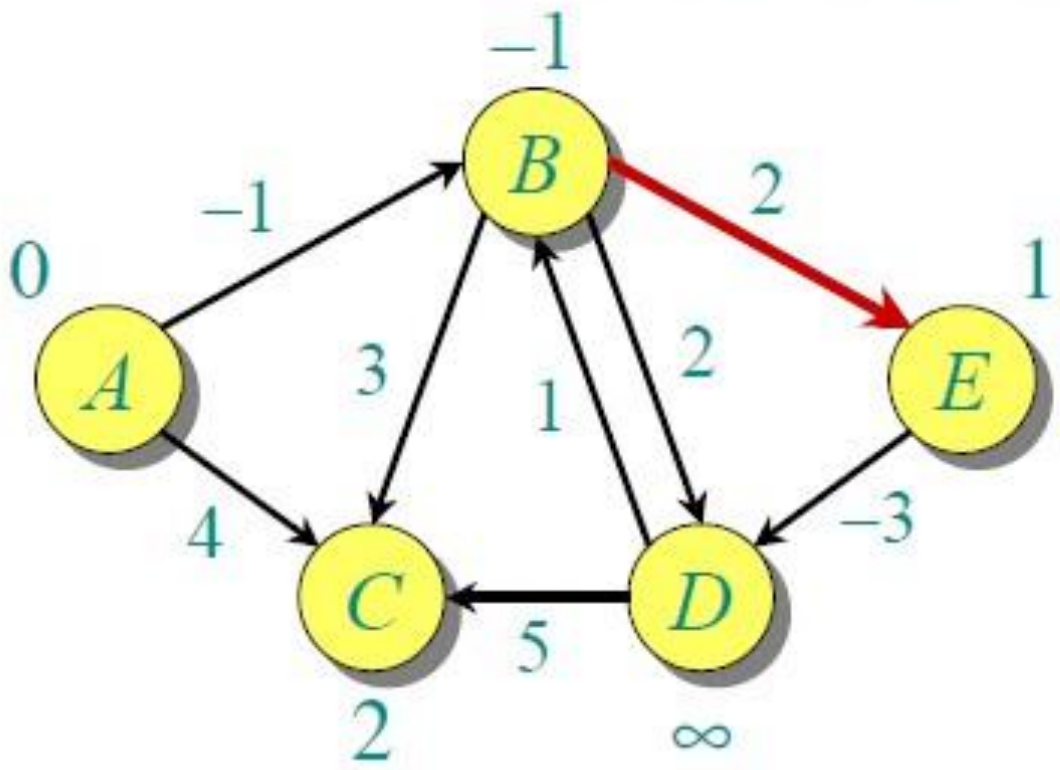


A	B	C	D	E
0	∞	∞	∞	∞
0	–1	∞	∞	∞
0	–1	4	∞	∞
0	–1	2	∞	∞

Pass 1

Example of Bellman-Ford

Order of edges: (B,E), (D,B), (B,D), (A,B), (A,C), (D,C), (B,C), (E,D)

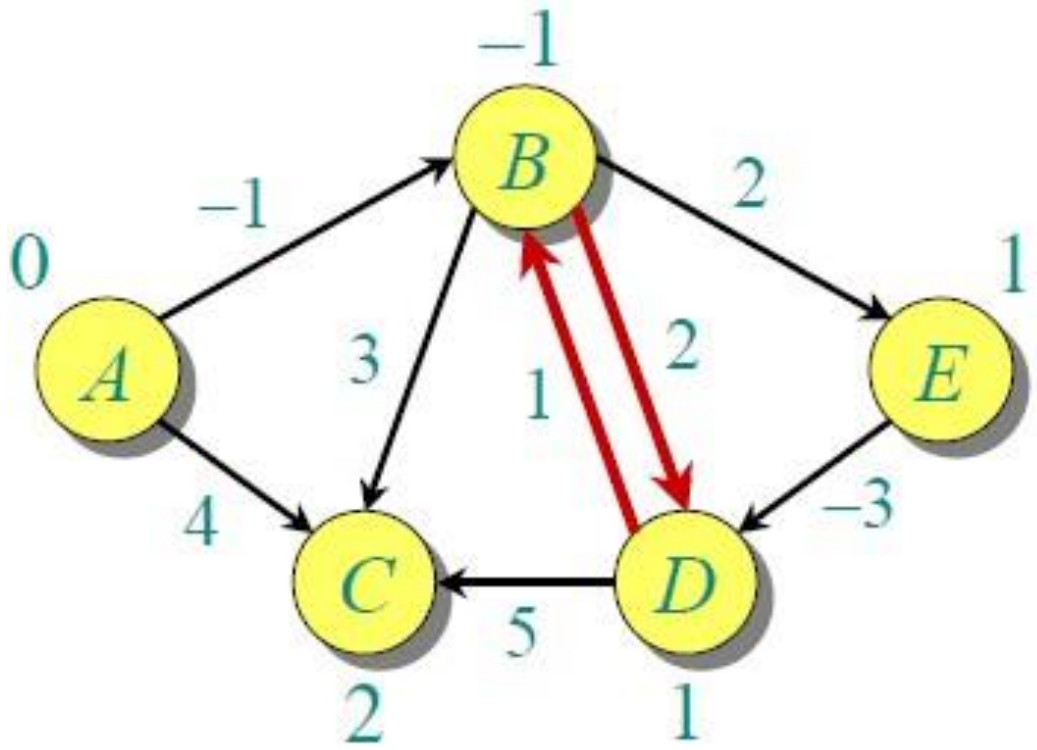


A	B	C	D	E
0	∞	∞	∞	∞
0	−1	∞	∞	∞
0	−1	4	∞	∞
0	−1	2	∞	∞
0	−1	2	∞	1

Pass 2

Example of Bellman-Ford

Order of edges: *(B,E), (D,B), (B,D), (A,B), (A,C), (D,C), (B,C), (E,D)*

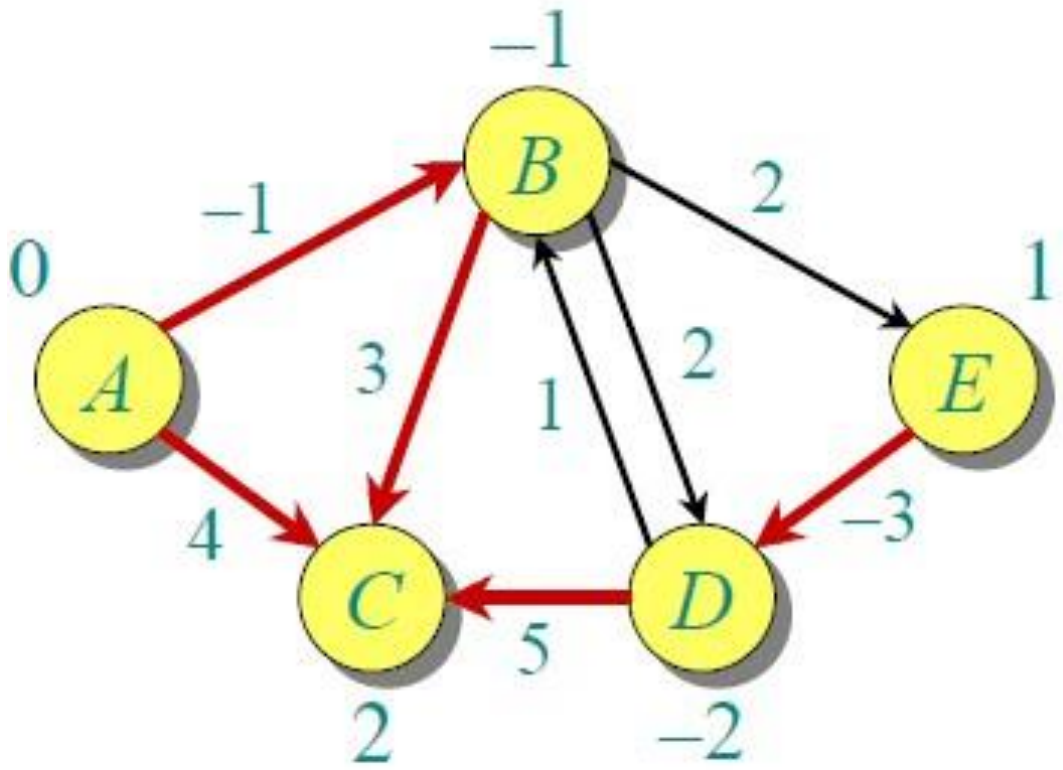


A	*B*	*C*	*D*	*E*
0	∞	∞	∞	∞
0	−1	∞	∞	∞
0	−1	4	∞	∞
0	−1	2	∞	∞
0	−1	2	∞	1
0	−1	2	1	1

Pass 2

Example of Bellman-Ford

Order of edges: *(B,E), (D,B), (B,D), (A,B), (A,C), (D,C), (B,C), (E,D)*

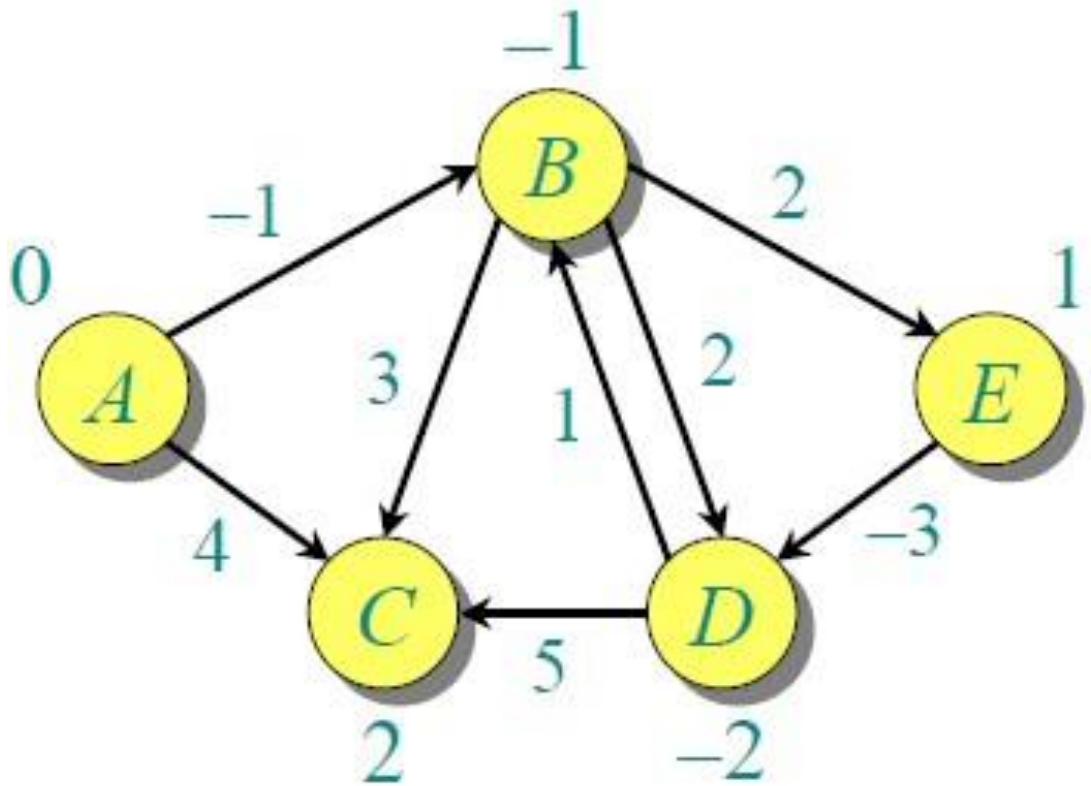


A	*B*	*C*	*D*	*E*
0	∞	∞	∞	∞
0	−1	∞	∞	∞
0	−1	4	∞	∞
0	−1	2	∞	∞
0	−1	2	∞	1
0	−1	2	1	1
0	−1	2	−2	1

Pass 2

Example of Bellman-Ford

Order of edges: *(B,E), (D,B), (B,D), (A,B), (A,C), (D,C), (B,C), (E,D)*



A	*B*	*C*	*D*	*E*
0	∞	∞	∞	∞
0	–1	∞	∞	∞
0	–1	4	∞	∞
0	–1	2	∞	∞
0	–1	2	∞	1
0	–1	2	1	1
0	–1	2	–2	1

Note: Values decrease monotonically.

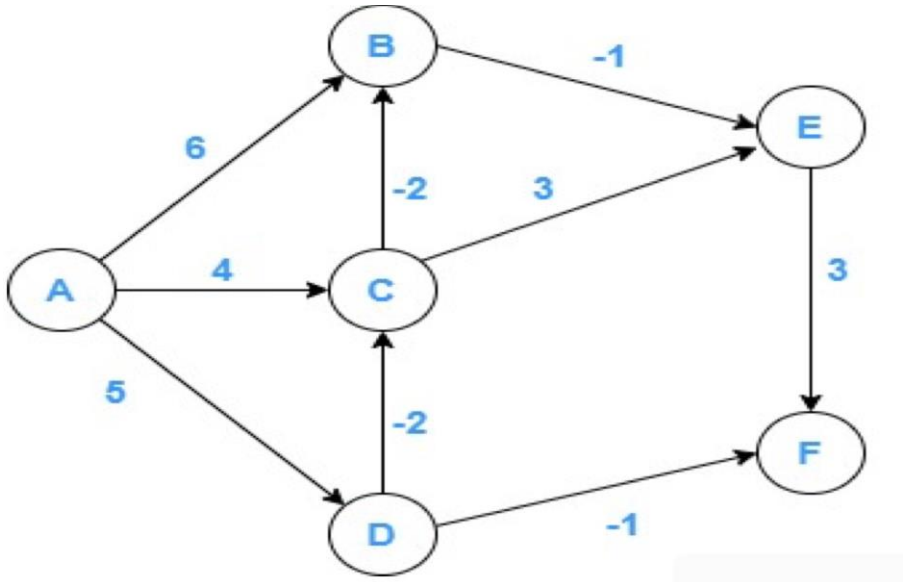
Bellman-Ford Algorithm

TryYourself

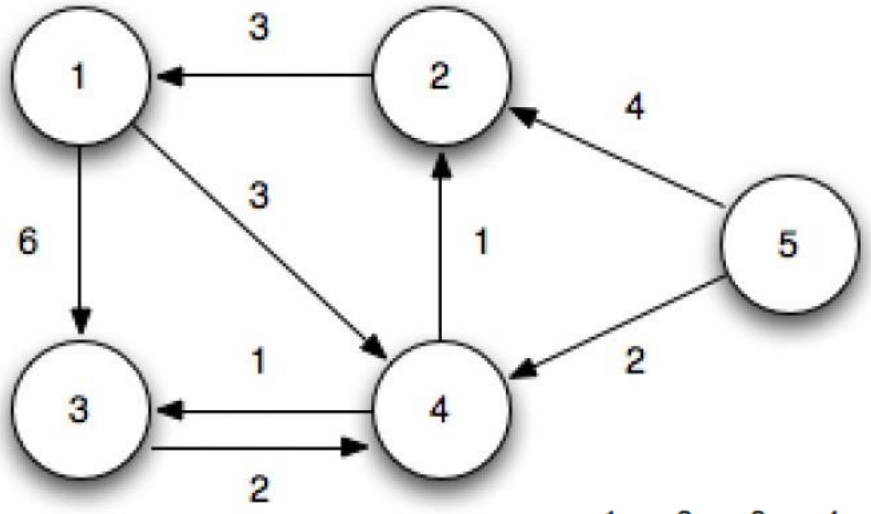
Find the shortest distance from source vertex to other vertex using Bellman-ford algorithm.

Answer

A- A: 0
A- B: 1
A- C: 3
A- D: 5
A- E: 0
A- F: 3



Source = A



Source = 5

	1	2	3	4	5
d	6	3	3	2	0
π	2	4	4	5	/

Advanced Shortest Path Algorithms

1. **Bellman-Ford Algorithm**
2. **Johnson's algorithm**

Shortest Path Algorithms

Johnson's algorithm (All Pair Shortest Path)

- Johnson's algorithm is a all pair shortest path algorithm i,e, it find the shortest path between every pair of vertices in that graph.
- It is similar to the Floyd-Warshall algorithm which is most effective for dense graphs (many edges), while Johnson's algorithm is most effective for sparse graphs (few edges).
- The reason that Johnson's algorithm is better for sparse graphs is that its time complexity depends on the number of edges in the graph. Johnson's algorithm runs in $\mathbf{O(V^2 \cdot \log(V) + |V| \cdot |E|)}$ time. So, if the number of edges is small (i.e. the graph is sparse), it will run faster than the $O(V^3)$ runtime of Floyd-Warshall.
- Johnson's algorithm uses two other shortest path algorithms as subroutines.
  - It uses Bellman-Ford in order to reweight the input graph to eliminate negative edges and detect negative cycles.
  - With the new graph, it then uses Dijkstra's shortest path algorithm to calculate the shortest path between all pairs of vertices.
  - The output of the algorithm is then the set of shortest paths in the original graph.

Johnson’s algorithm (All Pair Shortest Path for Sparse Graph)

Johnson's algorithm has three main steps.

1. A new vertex is added to the graph, and it is connected by edges of zero weight to all other vertices in the graph.
2. All edges go through a reweighting process that eliminates negative weight edges.
3. The added vertex from step 1 is removed and Dijkstra's algorithm is run on every node in the graph.

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Producing Nonnegative Weights by Reweighting

- Given a Weighted, Directed Graph $G = (V, E)$ with weight function $w : E \rightarrow R$, construct a new Graph $G' = (V', E')$ where $V' = V \cup \{s\}$ for some new vertex $s \notin V$ and $E' = E \cup \{(s, v) : v \in V\}$.
- The nonnegative weights $\hat{w}(u, v)$ are computed as follows:
  1. Extend the weight function w so that $w(s, v) = 0$ for all $v \in V$.
  2. Compute $h(v) = \delta(s, v)$ for all $v \in V'$.
  3. Compute $\hat{w}(u, v) = w(u, v) + h(u) - h(v) \geq 0$.

$h(v) \leq h(u) + w(u, v)$ (Triangle Inequality) (3.1)

$\hat{w}(u, v) = w(u, v) + h(u) - h(v)$ (Lemma) (3.2)

(3.1) and (3.2) $\rightarrow \hat{w}(u, v) = w(u, v) + h(u) - h(v) \geq 0$.

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Producing Nonnegative Weights by Reweighting...

1. $w(s, v) = 0$ for all $v \in V$.

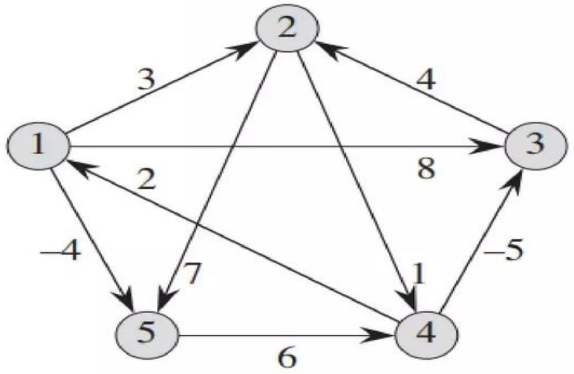


Fig (a): Graph G

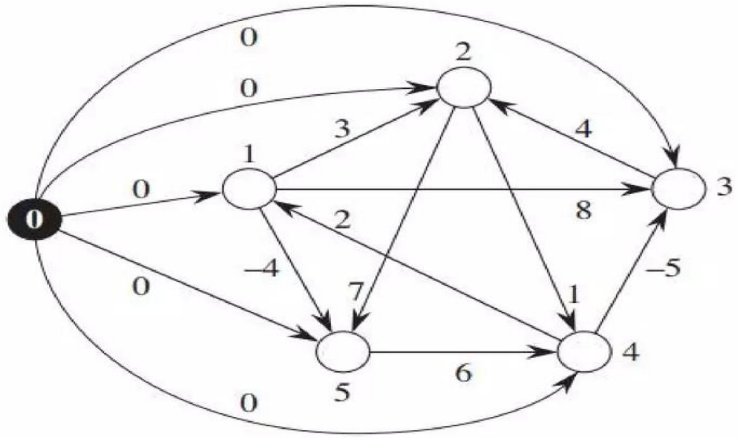


Fig (b): Graph G'

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Producing Nonnegative Weights by Reweighting...

2. $h(v) = \delta(s, v)$ for all $v \in V'$ (J1)

$\delta(s, v)$ is computed using Bellman-Ford Algorithm

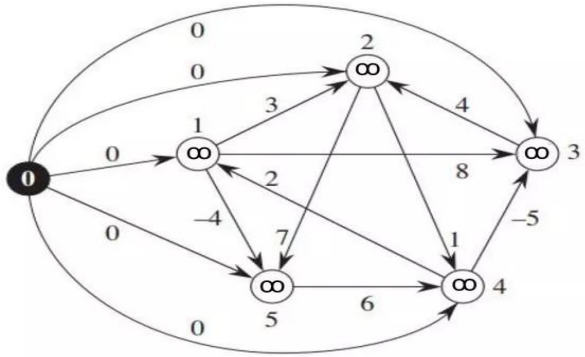
- The edges are relaxed in the following order:

  (0, 1); (0, 2); (0, 3); (0, 4); (0, 5); (1, 2); (1, 3); (1, 5); (2, 4); (2, 5); (3, 2); (4, 1); (4, 3); (5, 4);

- The Bellman-Ford Algorithm runs for 5 passes since there are 6 vertices in the graph G'
- At the end of 5th pass, $\delta(s, v)$, the shortest paths from s to all the vertices $v \in V'$ would be computed if G' does not consist a negative cycle.

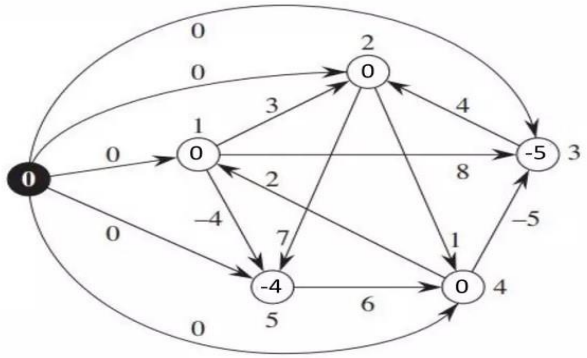
Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Producing Nonnegative Weights by Reweighting...



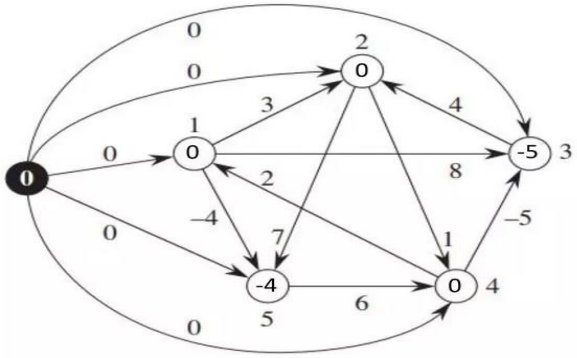
Relax (0, 1): $1.d = \min(1.d, 0.d + w(0, 1)) = (\infty, 0 + 0) = 0$
Relax (0, 2): $2.d = \min(2.d, 0.d + w(0, 2)) = (\infty, 0 + 0) = 0$
Relax (0, 3): $3.d = \min(3.d, 0.d + w(0, 3)) = (\infty, 0 + 0) = 0$
Relax (0, 4): $4.d = \min(4.d, 0.d + w(0, 4)) = (\infty, 0 + 0) = 0$
Relax (0, 5): $5.d = \min(5.d, 0.d + w(0, 5)) = (\infty, 0 + 0) = 0$
Relax (1, 2): $2.d = \min(2.d, 1.d + w(1, 2)) = (0, 0 + 3) = 0$
Relax (1, 3): $3.d = \min(3.d, 1.d + w(1, 3)) = (0, 0 + 8) = 0$

Relax (1, 5): $5.d = \min(5.d, 1.d + w(1, 5)) = (0, 0 + (-4)) = -4$
Relax (2, 4): $4.d = \min(4.d, 2.d + w(2, 4)) = (0, 0 + 1) = 0$
Relax (2, 5): $5.d = \min(5.d, 2.d + w(2, 5)) = (-4, 0 + 7) = -4$
Relax (3, 2): $2.d = \min(2.d, 3.d + w(3, 2)) = (0, 0 + 4) = 0$
Relax (4, 1): $1.d = \min(1.d, 4.d + w(4, 1)) = (0, 0 + 2) = 0$
Relax (4, 3): $3.d = \min(3.d, 4.d + w(4, 3)) = (0, 0 + -5) = -5$
Relax (5, 4): $4.d = \min(4.d, 5.d + w(5, 4)) = (0, (-4) + 6) = 0$



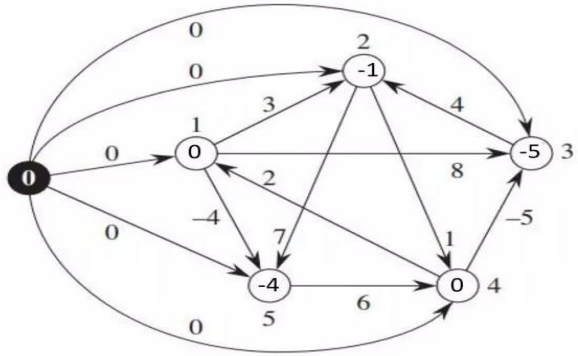
Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Producing Nonnegative Weights by Reweighting...

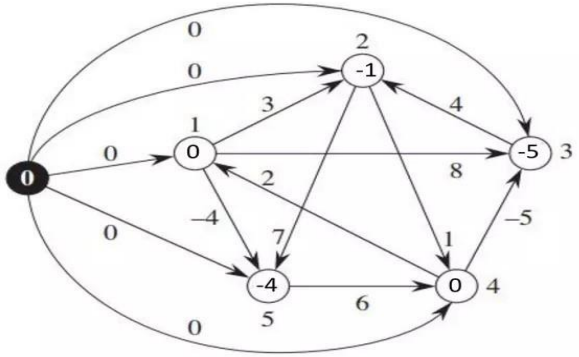


Relax (0, 1): $1.d = \min(1.d, 0.d + w(0, 1)) = (0, 0 + 0) = 0$
Relax (0, 2): $2.d = \min(2.d, 0.d + w(0, 2)) = (0, 0 + 0) = 0$
Relax (0, 3): $3.d = \min(3.d, 0.d + w(0, 3)) = (-5, 0 + 0) = -5$
Relax (0, 4): $4.d = \min(4.d, 0.d + w(0, 4)) = (0, 0 + 0) = 0$
Relax (0, 5): $5.d = \min(5.d, 0.d + w(0, 5)) = (-4, 0 + 0) = -4$
Relax (1, 2): $2.d = \min(2.d, 1.d + w(1, 2)) = (0, 0 + 3) = 0$
Relax (1, 3): $3.d = \min(3.d, 1.d + w(1, 3)) = (-5, 0 + 8) = -5$

Relax (1, 5): $5.d = \min(5.d, 1.d + w(1, 5)) = (-4, 0 + (-4)) = -4$
Relax (2, 4): $4.d = \min(4.d, 2.d + w(2, 4)) = (0, 0 + 1) = 0$
Relax (2, 5): $5.d = \min(5.d, 2.d + w(2, 5)) = (-4, 0 + 7) = -4$
Relax (3, 2): $2.d = \min(2.d, 3.d + w(3, 2)) = (0, (-5) + 4) = -1$
Relax (4, 1): $1.d = \min(1.d, 4.d + w(4, 1)) = (0, 0 + 2) = 0$
Relax (4, 3): $3.d = \min(3.d, 4.d + w(4, 3)) = (-5, 0 + -5) = -5$
Relax (5, 4): $4.d = \min(4.d, 5.d + w(5, 4)) = (0, (-4) + 6) = 0$



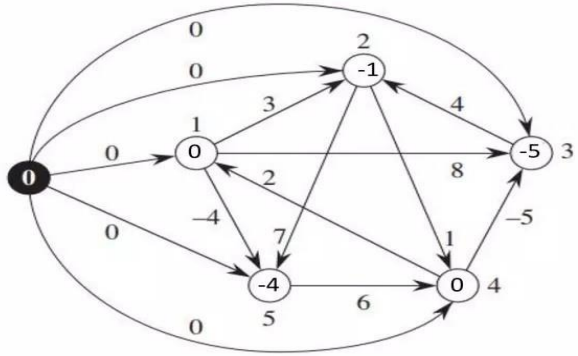
Johnson's algorithm (All Pair Shortest Path for Sparse Graph)



Producing Nonnegative Weights by Reweighting...

Relax (0, 1): $1.d = \min(1.d, 0.d + w(0, 1)) = (0, 0 + 0) = 0$
Relax (0, 2): $2.d = \min(2.d, 0.d + w(0, 2)) = (-1, 0 + 0) = -1$
Relax (0, 3): $3.d = \min(3.d, 0.d + w(0, 3)) = (-5, 0 + 0) = -5$
Relax (0, 4): $4.d = \min(4.d, 0.d + w(0, 4)) = (0, 0 + 0) = 0$
Relax (0, 5): $5.d = \min(5.d, 0.d + w(0, 5)) = (-4, 0 + 0) = -4$
Relax (1, 2): $2.d = \min(2.d, 1.d + w(1, 2)) = (-1, 0 + 3) = 0$
Relax (1, 3): $3.d = \min(3.d, 1.d + w(1, 3)) = (-5, 0 + 8) = -5$

Relax (1, 5): $5.d = \min(5.d, 1.d + w(1, 5)) = (-4, 0 + (-4)) = -4$
Relax (2, 4): $4.d = \min(4.d, 2.d + w(2, 4)) = (-1, 0 + 1) = -1$
Relax (2, 5): $5.d = \min(5.d, 2.d + w(2, 5)) = (-4, (-1) + 7) = -4$
Relax (3, 2): $2.d = \min(2.d, 3.d + w(3, 2)) = (-1, (-5) + 4) = -1$
Relax (4, 1): $1.d = \min(1.d, 4.d + w(4, 1)) = (0, 0 + 2) = 0$
Relax (4, 3): $3.d = \min(3.d, 4.d + w(4, 3)) = (-5, 0 + (-5)) = -5$
Relax (5, 4): $4.d = \min(4.d, 5.d + w(5, 4)) = (0, (-4) + 6) = 0$



Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Producing Nonnegative Weights by Reweighting…

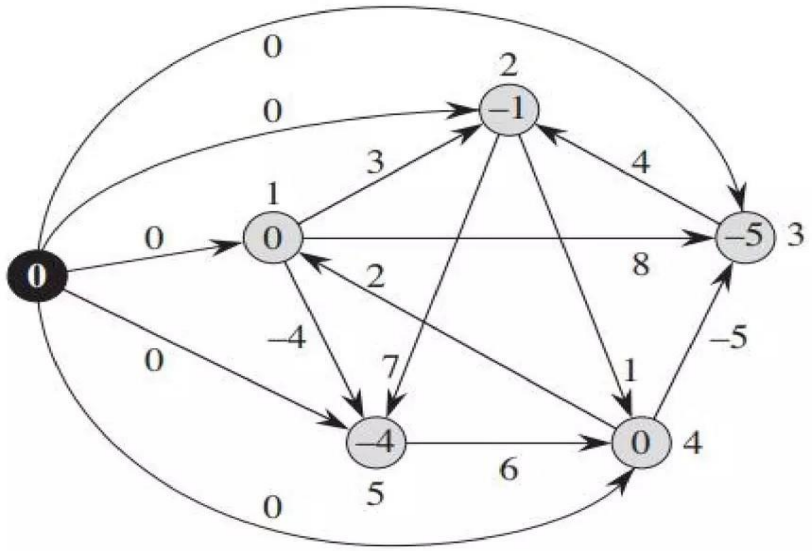


Fig (*c*): *h* (*v*) computed for *G* ′ of Fig (*b*)

Vertex, *v*	*h* (*v*)
1	0
2	-1
3	-5
4	0
5	-4

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Producing Nonnegative Weights by Reweighting...

3. $\hat{w}(u, v) = w(u, v) + h(u) - h(v) \geq 0$

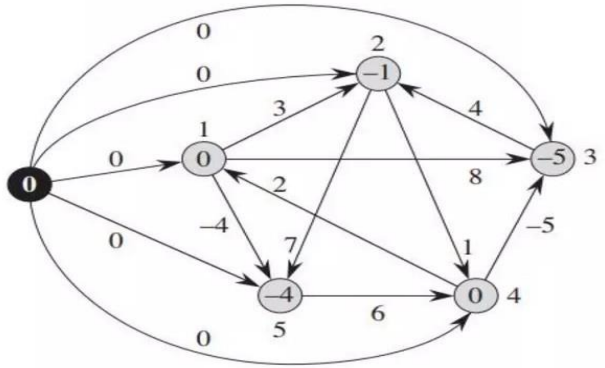


Fig (c): $h(v)$ computed for G'

$\hat{w}(5,4) = w(5,4) + h(5) - h(4)=6-4-0=2$
$\hat{w}(2,4) = w(2,4) + h(2) - h(4)=1-1-0=0$
$\hat{w}(4,3) = w(4,3) + h(4) - h(3)=-5+0+5=0$
$\hat{w}(3,2) = w(3,2) + h(3) - h(2)=4-5+1=0$

$\hat{w}(1,2) = w(1,2) + h(1) - h(2)=3+0+1=4$
$\hat{w}(1,3) = w(1,3) + h(1) - h(3)=8+0+5=13$
$\hat{w}(4,1) = w(4,1) + h(4) - h(1)=2+0-0=2$
$\hat{w}(1,5) = w(1,5) + h(1) - h(5)=-4+0+4=0$
$\hat{w}(2,5) = w(2,5) + h(2) - h(5)=7-1+4=10$

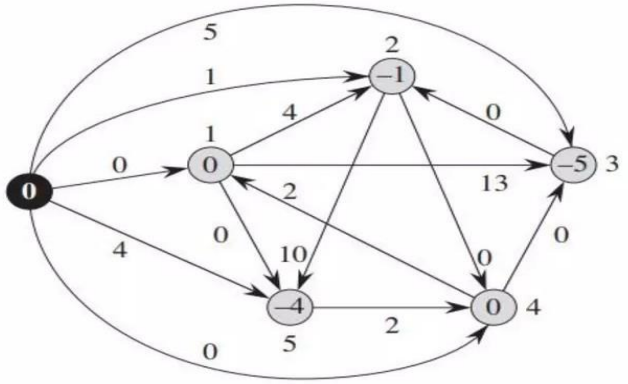


Fig (d): G' with Reweighted edges

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Computing All-Pairs Shortest Path

- Dijkstra's algorithm is used to find the shortest paths.
- The algorithm is run once with each vertex $v \in V$ as source.
- It takes the reweighted graph G' as input and computes $\hat{\delta}$ for all the vertex $v \in V$.
- Finally $\delta(u, v) = \hat{\delta}(u, v) + h(v) - h(u)$
- After $V - 1$ passes the shortest paths between all pairs of vertices would be computed.

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Algorithm

- Johnson's algorithm uses as subroutines both the Bellman-Ford algorithm and the Dijkstra's algorithm.
- At first, it extends the given weighted directed graph G to produce graph G'.
- Runs Bellman-Ford algorithm to compute $h(v)$ if the graph G' does not contain negative cycles.
- Reweights the graph G' to get reweighted edges E'.
- Runs Dijkstra's algorithm $V - 1$ times to get All-Pairs Shortest Path of G', and correspondingly computes the shortest paths of the graph G using: $\delta(u, v) = \hat{\delta}(u, v) + h(v) - h(u)$
- It returns a $|V| \times |V|$ matrix, $D = d_{uv}$, where $d_{uv} = \delta(u, v)$.

Running Time:

- If the min-priority queue in Dijkstra's algorithm is implemented by a Fibonacci heap, Johnson's algorithm runs in $O(V^2 \lg V + VE)$ time.
- Binary min heap implementation yields a running time of $O(VE \lg V)$.

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Algorithm...

JOHNSON (G, w)

Compute G', where $G'.V = G.V \cup \{s\}$, $G'.E = G.E \cup \{(s, v): v \in G.V\}$ and $w(s, v) = 0$ for all $v \in G.V$

If (BELLMAN-FORD (G', w, s) == *FALSE*)

    print "Graph contains Negative-Weight Cycle"

Else For (Each Vertex $v \in G'.V$)

    Set $h(v)$ to the value of $\delta(s, v)$ computed by BELLMAN-FORD algorithm

    For each edge $(u, v) \in G'.E$

        $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$

    Let $D = (d_{uv})$ be a new n x n matrix

    For (Each Vertex $u \in G.V$)

        Run DIJKSTRA (G, $\hat{w}$, u) to compute $\hat{\delta}(u, v)$ for all $v \in G.V$

        For (Each Vertex $v \in G.V$)

            $d_{uv} = \hat{\delta}(u, v) + h(v) - h(u)$

    Return D

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

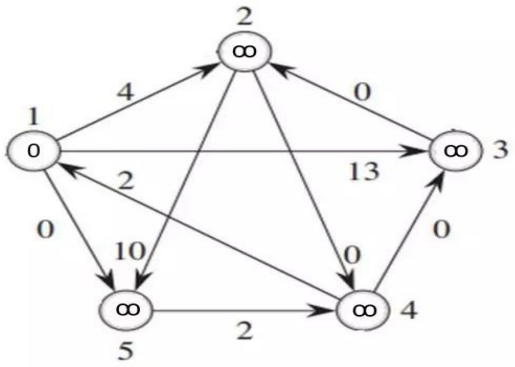


Fig (*e*): Reweighted Graph

Example

$\hat{\delta}$ (*u*, *v*) from Source 1

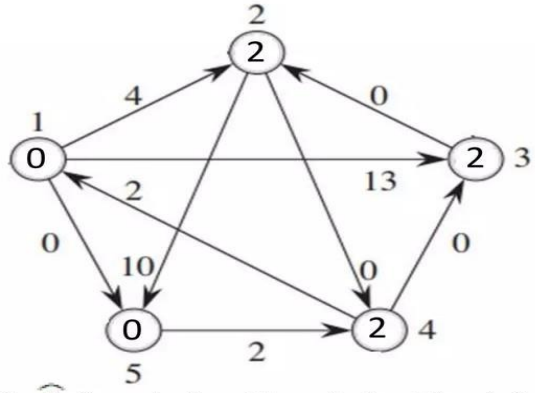


Fig (*f*): $\hat{\delta}$ (*u*, *v*) for Graph in Fig (*e*) from Source 1using Dijkstra's Algorithm

S	*V*	Relax (*u*, *v*, *w*)	*Q*	*u*	Cost
			1,2,3,4,5	1	0
1	2 3 5	Relax (1,2): $2.d = \min(2.d, 1.d + w(1, 2)) = (\infty, 0 + 4) = 4$ Relax (1,3): $3.d = \min(3.d, 1.d + w(1, 3)) = (\infty, 0 + 13) = 13$ Relax (1,5): $5.d = \min(5.d, 1.d + w(1, 5)) = (\infty, 0 + 0) = 0$	2,3,4,5	5	0
1,5	4	Relax (5,4): $4.d = \min(4.d, 5.d + w(5, 4)) = (\infty, 0 + 2) = 2$	2,3,4	4	2
1,5,4	3	Relax (4,3): $3.d = \min(3.d, 4.d + w(4, 3)) = (13, 2 + 0) = 2$	2,3	3	2
1,5,4,3	2	Relax (3,2): $2.d = \min(2.d, 3.d + w(3, 2)) = (4, 2 + 0) = 2$	2	2	2

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Example...

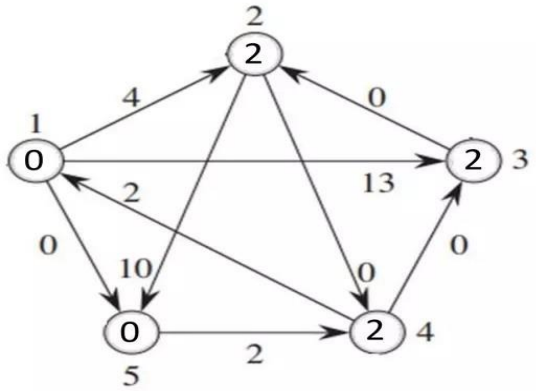


Fig (*f*): $\hat{\delta}(u, v)$ from Source 1

$\delta(u, v)$ from Source 1

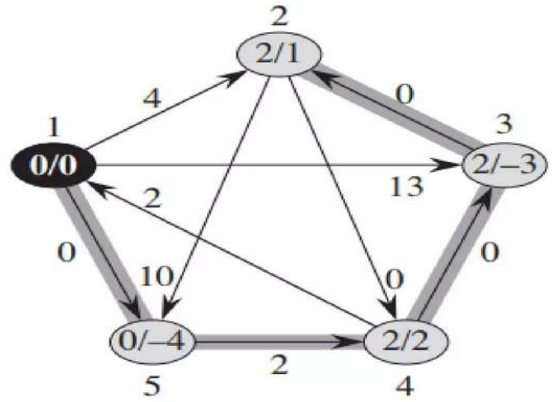


Fig (*g*): δ (*u*, *v*) of Graph in Fig (*f*) from source 1 using $d_{uv} = \hat{\delta}(u, v) + h(v) - h(u)$
($\hat{\delta}$ (u,v), δ (u,v)) – Values within each vertex.

$$\delta(1, 2) = \hat{\delta}(1,2) + h(2) - h(1) = 2 - 1 - 0 = 1$$
$$\delta(1, 3) = \hat{\delta}(1,3) + h(3) - h(1) = 2 - 5 - 0 = -3$$
$$\delta(1, 4) = \hat{\delta}(1,4) + h(4) - h(1) = 2 + 0 - 0 = 2$$
$$\delta(1, 5) = \hat{\delta}(1,5) + h(5) - h(1) = 0 - 4 - 0 = -4$$

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

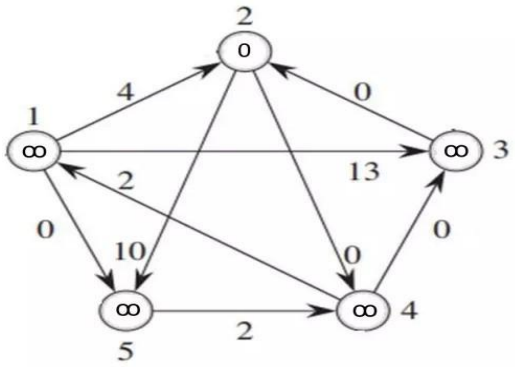


Fig (*e*): Reweighted Graph

$\widehat{\delta}\ (u, v)$ from Source 2

Example...

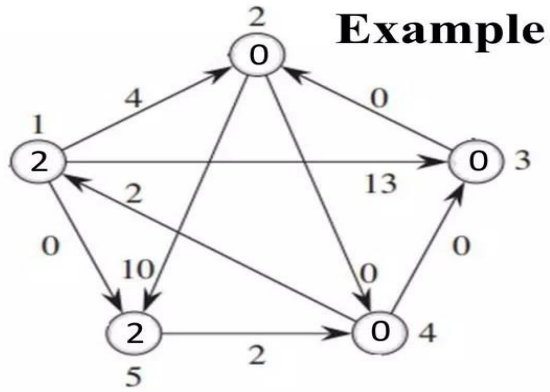
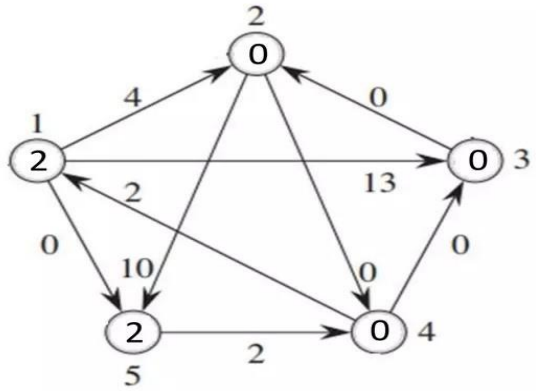


Fig (*h*): $\widehat{\delta}\ (u, v)$ for Graph in Fig (*e*) from Source 2 using Dijkstra's Algorithm

S	*V*	Relax (*u*, *v*, *w*)	*Q*	*u*	Cost
			1,2,3,4,5	2	0
2	4 5	Relax (2, 4): $4.d = \min(4.d, 2.d + w(2, 4)) = (\infty, 0 + 0) = 0$ Relax (2, 5): $5.d = \min(5.d, 2.d + w(2, 5)) = (\infty, 0 + 10) = 10$	1,3,4,5	4	0
2,4	1 3	Relax (4, 1): $1.d = \min(1.d, 4.d + w(4, 1)) = (\infty, 0 + 2) = 2$ Relax (4, 3): $3.d = \min(3.d, 4.d + w(4, 3)) = (\infty, 0 + 0) = 0$	1,3,5	3	0
2,4,3	-		1,5	1	2
2,4,3,1	5	Relax (1, 5): $5.d = \min(5.d, 1.d + w(1, 5)) = (10, 2 + 0) = 2$	5	5	2

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Example...



δ (u, v) from Source 2

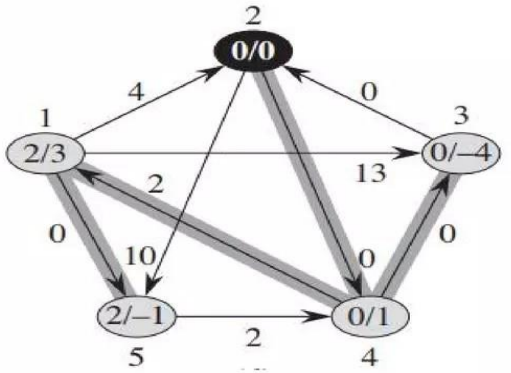


Fig (*h*): $\hat{\delta}$ (*u*, *v*) from Source 2

Fig (*i*): δ (*u*, *v*) of Graph in Fig (*f*) from source 2 using $d_{uv} = \hat{\delta}(u, v) + h(v) - h(u)$
($\hat{\delta}$ (u,v), δ (u,v)) – Values within each vertex.

$$\delta(2, 1) = \hat{\delta}(2, 1) + h(1) - h(2) = 2 + 0 + 1 = 3$$
$$\delta(2, 3) = \hat{\delta}(2, 3) + h(3) - h(2) = 0 - 5 + 1 = -4$$
$$\delta(2, 4) = \hat{\delta}(2, 4) + h(4) - h(2) = 0 + 0 + 1 = 1$$
$$\delta(2, 5) = \hat{\delta}(2, 5) + h(5) - h(2) = 2 - 4 + 1 = -1$$

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

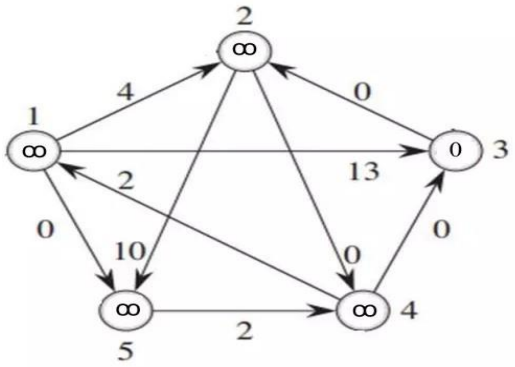


Fig (*e*): Reweighted Graph

$\widehat{\delta}\ (u, v)$ from Source 3

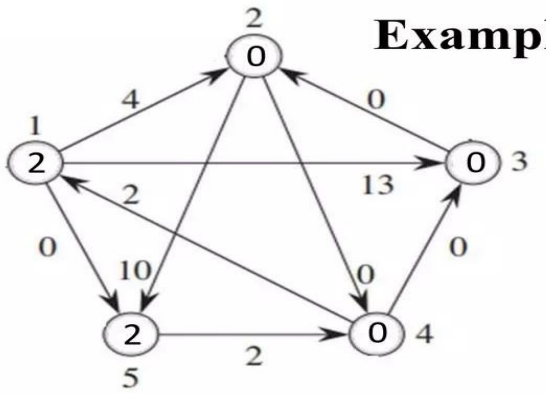
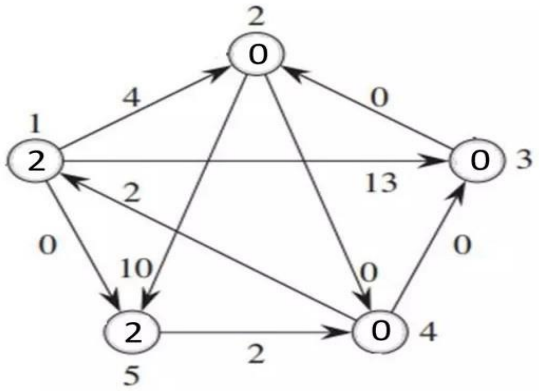


Fig (*j*): $\widehat{\delta}\ (u, v)$ for Graph in Fig (*e*) from Source 3 using Dijkstra's Algorithm

S	*V*	Relax (*u*, *v*, *w*)	*Q*	*u*	Cost
			1,2,3,4,5	3	0
3	2	Relax (3, 2): $2.d = \min(2.d, 3.d + w(3, 2)) = (\infty, 0 + 0) = 0$	1,2,4,5	2	0
3,2	4 5	Relax (2, 4): $4.d = \min(4.d, 2.d + w(2, 4)) = (\infty, 0 + 0) = 0$ Relax (2, 5): $5.d = \min(5.d, 2.d + w(2, 5)) = (\infty, 0 + 10) = 10$	1,4,5	4	0
3,2,4	1	Relax (4, 1): $1.d = \min(1.d, 4.d + w(4, 1)) = (\infty, 0 + 2) = 2$	1,5	1	2
3,2,4,1	5	Relax (1, 5): $5.d = \min(5.d, 1.d + w(1, 5)) = (10, 2 + 0) = 2$	5	5	2

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Example...



$\delta\ (u, v)$ from Source 3

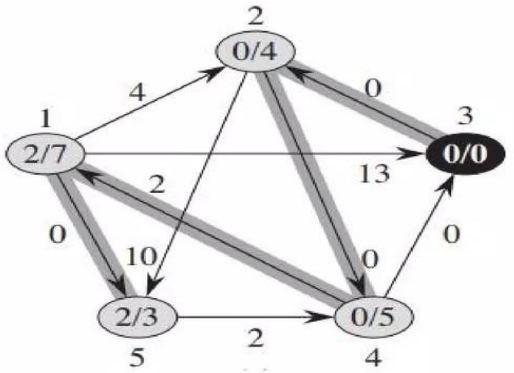


Fig (*j*): $\widehat{\delta}\ (u, v)$ from Source 3

Fig (*k*): $\delta\ (u, v)$ of Graph in Fig (*f*) from source 3 using $d_{uv} = \widehat{\delta}\ (u, v) + h\ (v) - h\ (u)$
($\widehat{\delta}$ (u,v), δ (u,v)) – Values within each vertex.

$$\delta\ (3, 1) = \widehat{\delta}\ (3, 1) + h\ (1) - h\ (3) = 2 + 0 + 5 = 7$$
$$\delta\ (3, 2) = \widehat{\delta}\ (3, 2) + h\ (2) - h\ (3) = 0 - 1 + 5 = 4$$
$$\delta\ (3, 4) = \widehat{\delta}\ (3, 4) + h\ (4) - h\ (3) = 0 + 0 + 5 = 5$$
$$\delta\ (3, 5) = \widehat{\delta}\ (3, 5) + h\ (5) - h\ (3) = 2 - 4 + 5 = 3$$

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

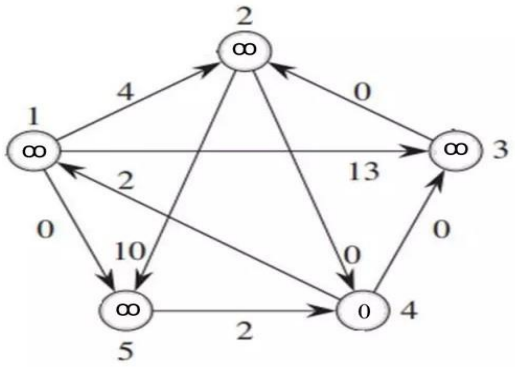


Fig (*e*): Reweighted Graph

$\hat{\delta}$ (*u*, *v*) from Source 4

Example...

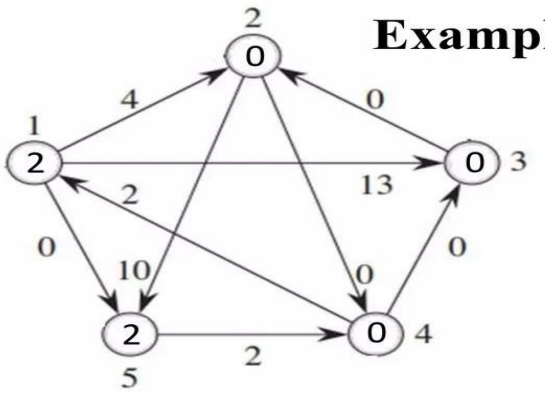
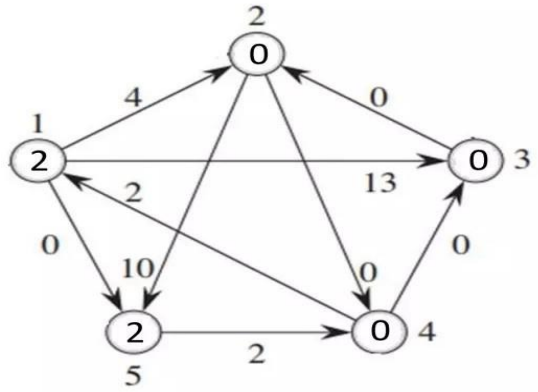


Fig (*l*): $\hat{\delta}$ (*u*, *v*) for Graph in Fig (*e*) from Source 4 using Dijkstra's Algorithm

S	*V*	Relax (*u*, *v*, *w*)	*Q*	*u*	Cost
			1,2,3,4,5	4	0
4	1 3	Relax (4, 1): $1.d = \min(1.d, 4.d + w(4, 1)) = (\infty, 0 + 2) = 2$ Relax (4, 3): $3.d = \min(3.d, 4.d + w(4, 3)) = (\infty, 0 + 0) = 0$	1,2,3,5	3	0
4,3	2	Relax (3, 2): $2.d = \min(2.d, 3.d + w(3, 2)) = (\infty, 0 + 0) = 0$	1,2,5	2	0
4,3,2	5	Relax (2, 5): $5.d = \min(5.d, 2.d + w(2, 5)) = (\infty, 0 + 10) = 10$	1,5	1	2
4,3,2,1	5	Relax (1, 5): $5.d = \min(5.d, 1.d + w(1, 5)) = (10, 2 + 0) = 2$	5	5	2

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Example...



$\delta\,(u, v)$ from Source 4

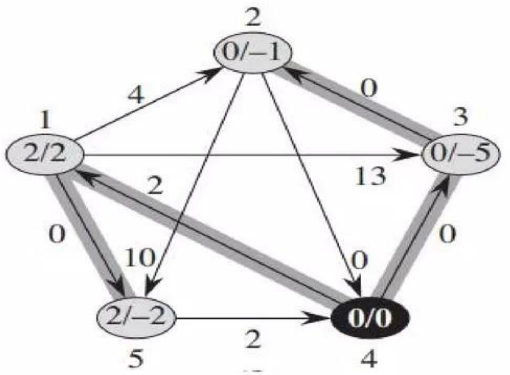


Fig (l): $\hat{\delta}\,(u, v)$ from Source 4

Fig (m): $\delta\,(u, v)$ of Graph in Fig (f) from source 4 using $d_{uv} = \hat{\delta}\,(u, v) + h\,(v) - h\,(u)$
($\hat{\delta}$ (u,v), δ (u,v)) – Values within each vertex.

$$\delta\,(4, 1) = \hat{\delta}\,(4, 1) + h\,(1) - h\,(4) = 2 + 0 - 0 = 2$$
$$\delta\,(4, 2) = \hat{\delta}\,(4, 2) + h\,(2) - h\,(4) = 0 - 1 - 0 = -1$$
$$\delta\,(4, 3) = \hat{\delta}\,(4, 3) + h\,(3) - h\,(4) = 0 - 5 - 0 = -5$$
$$\delta\,(4, 5) = \hat{\delta}\,(4, 5) + h\,(5) - h\,(4) = 2 - 4 - 0 = -2$$

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

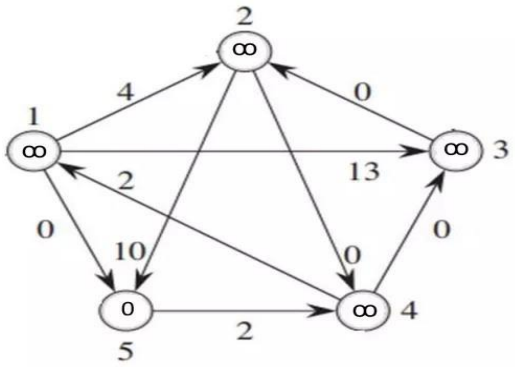


Fig (*e*): Reweighted Graph

$\widehat{\delta}\ (u, v)$ from Source 5

Example...

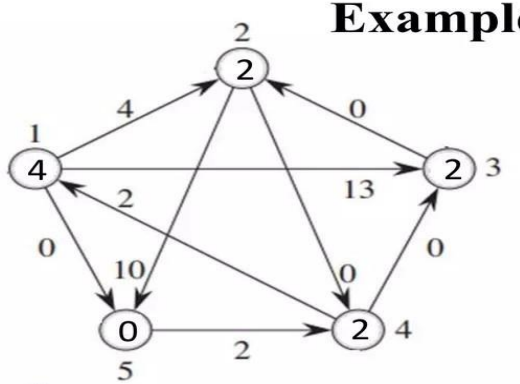


Fig (*n*): $\widehat{\delta}\ (u, v)$ for Graph in Fig (*e*) from Source 5 using Dijkstra's Algorithm

S	*V*	Relax (*u*, *v*, *w*)	*Q*	*u*	Cost
			1,2,3,4,5	5	0
5	4	Relax (5, 4): $4.d = \min(4.d, 5.d + w(5, 4)) = (\infty, 0 + 2) = 2$	1,2,3,4	4	2
5,4	1 3	Relax (4, 1): $1.d = \min(1.d, 4.d + w(4, 1)) = (\infty, 2 + 2) = 4$ Relax (4, 3): $3.d = \min(3.d, 4.d + w(4, 3)) = (\infty, 2 + 0) = 2$	1,2,3	3	2
5,4,3	2	Relax (3, 2): $2.d = \min(2.d, 3.d + w(3, 2)) = (\infty, 2 + 0) = 2$	1,2	2	2
5,4,3,2	-		1	1	4

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Example...

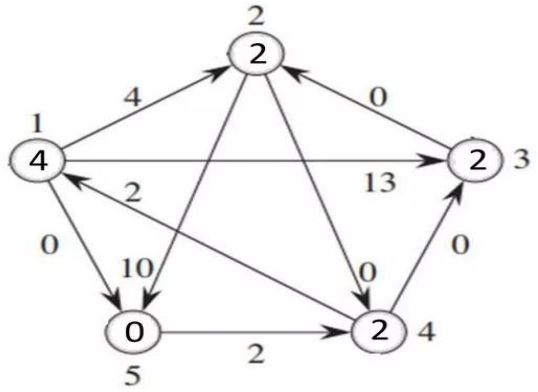


Fig (*n*): $\hat{\delta}$ (*u*, *v*) from Source 5

δ (*u*, *v*) from Source 5

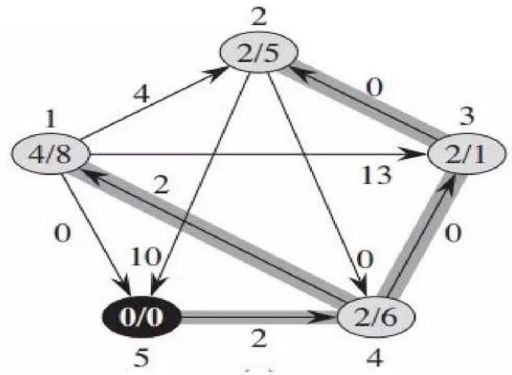


Fig (*o*): δ (*u*, *v*) of Graph in Fig (*f*) from source 5 using $d_{uv} = \hat{\delta}$ (*u*, *v*) + *h* (*v*) – *h* (*u*)
($\hat{\delta}$ (u,v), δ (u,v)) – Values within each vertex.

$$\delta(5, 1) = \hat{\delta}(5, 1) + h(1) - h(5) = 4 + 0 + 4 = 8$$
$$\delta(5, 2) = \hat{\delta}(5, 2) + h(2) - h(5) = 2 - 1 + 4 = 5$$
$$\delta(5, 3) = \hat{\delta}(5, 3) + h(3) - h(5) = 2 - 5 + 4 = 1$$
$$\delta(5, 4) = \hat{\delta}(5, 4) + h(4) - h(5) = 2 + 0 + 4 = 6$$

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Example...

Output:

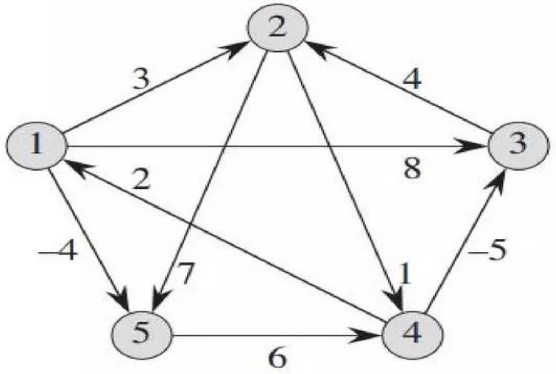


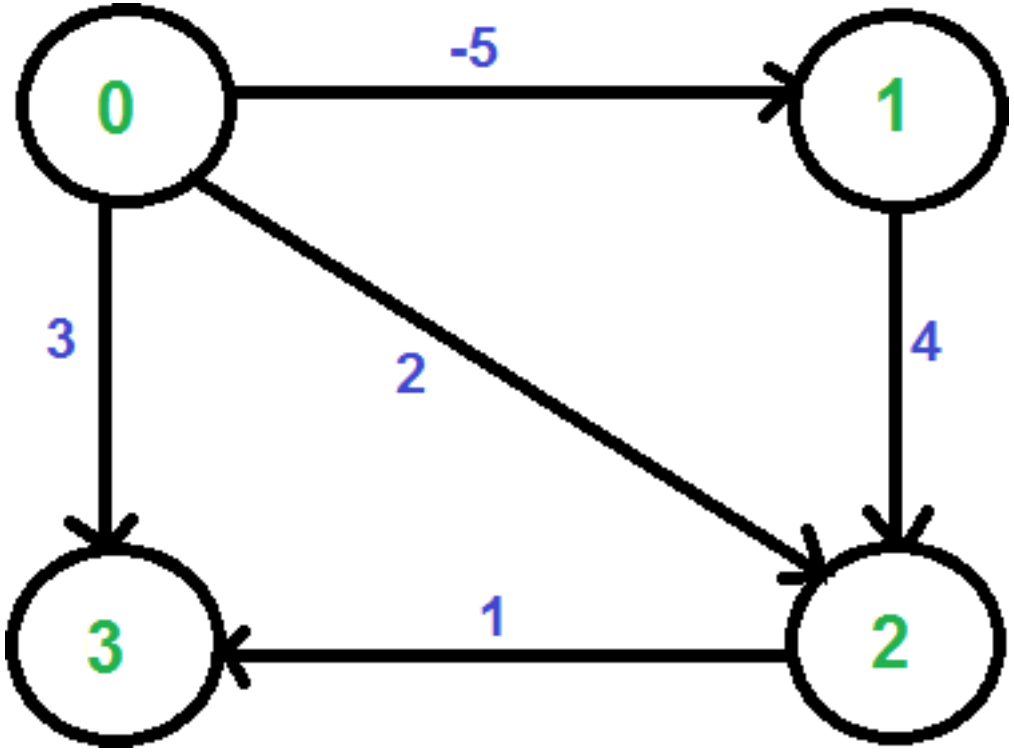
Fig (a): Input Graph G

All-Pairs Shortest Path Matrix d_{uv}

	1	2	3	4	5
1	0	1	-3	2	-4
2	3	0	-4	1	-1
3	7	4	0	5	3
4	2	-1	-5	0	-2
5	8	5	1	6	0

Try yourself

- Find the all pair shortest path using Johnson Algorithm for below figure



Answer:

	0	1	2	3
0	0	-5	-1	0
1	INF	0	4	5
2	INF	INF	0	1
3	INF	INF	INF	0